World Models in Action

Table of Contents

3. Representation learning before generation . . . 1
- 3.1 Learning a state without labels . . . 3
  - 3.1.1 Aliasing: when compression loses the wrong information . . . 4
  - 3.1.2 Why not train directly on the state? . . . 6
  - 3.1.3 The objective determines what the encoder is rewarded for keeping . . . 8
  - 3.1.4 Why use a benchmark rather than a toy world? . . . 9
  - 3.1.5 The experimental contract . . . 9
- 3.2 Pretext tasks: supervision from structure . . . 11
  - 3.2.1 Rotation prediction . . . 11
  - 3.2.2 Two other pretext tasks . . . 12
  - 3.2.3 Why a correct pretext answer may still be the wrong goal . . . 13
- 3.3 Contrastive learning from first principles . . . 14
  - 3.3.1 Views, positives, and negatives . . . 14
  - 3.3.2 From similarity to a loss . . . 16
  - 3.3.3 What InfoNCE actually optimizes . . . 18
  - 3.3.4 Training a contrastive encoder . . . 20
  - 3.3.5 Reading the state back with a linear probe . . . 24
- 3.4 The collapse problem and how it is avoided . . . 27
  - 3.4.1 Demonstrating collapse . . . 27
  - 3.4.2 Three mechanisms in brief . . . 30
  - 3.4.3 VICReg: making the anti-collapse force explicit . . . 34
- 3.5 Masked prediction . . . 38
  - 3.5.1 From masked words to masked patches . . . 38
  - 3.5.2 Building a masked autoencoder . . . 38
  - 3.5.3 Reconstruction versus representation . . . 40
- 3.6 Latent bottlenecks and compact state . . . 43
  - 3.6.1 The information bottleneck as a lens . . . 43
  - 3.6.2 Choosing the latent dimension . . . 45
- 3.7 Predicting over time: from invariance to prediction . . . 48
  - 3.7.1 Why the future is not just another view . . . 48
  - 3.7.2 Contrastive predictive coding . . . 49
  - 3.7.3 CPC evaluation: future retrieval and state probes . . . 52
- 3.8 Assembling the encoder . . . 55
  - 3.8.1 A common encoder interface . . . 55
  - 3.8.2 Verifying the contract . . . 57
  - 3.8.3 What each objective pays to preserve . . . 57

3.8.4 Checking the Chapter 2 contracts . . . 59
Exercises . . . 60
Where the bound comes from . . . 62
Why this bound is not a physical-state guarantee . . . 63
Self-supervised objectives . . . 64
Implementation and reproducibility . . . 64
Figure credits . . . 64
Updated taxonomy sources . . . 65

3. Representation learning before generation

This chapter covers

- Learning representations without labels
- Contrastive learning, derived and trained
- Collapse and its remedies
- Masked prediction and latent bottlenecks
- Measuring and assembling a state representation

Chapter 1 introduced the three components of a world model: an encoder that compresses observations into a state, a dynamics model that predicts how the state changes, and a planner that evaluates possible futures. Chapter 2 examined what makes a useful state. It must retain the information needed to predict what happens next, avoid carrying unnecessary detail, and distinguish situations that lead to different futures. But how do we train an encoder to meet these requirements when its input is a stream of pixels and nobody has labeled which details matter?

This chapter develops one answer: self-supervised learning, in which a model learns from training signals derived from the data itself rather than from human-provided labels. An image contains more structure than its pixel values alone suggest. Different views may depict the same underlying scene. A missing region may be predictable from its surroundings. An earlier observation may contain clues about what comes next. Each relationship provides a possible learning signal, and each encourages the encoder to retain a different kind of information.

That difference is the central idea of this chapter: a learning objective specifies what information a representation is encouraged to retain. A contrastive objective rewards features that remain consistent across selected views. Masked prediction rewards features that help reconstruct missing parts of an observation. Temporal prediction rewards features that help anticipate later observations. None of these objectives automatically guarantees a good state for a world model. The important question is not simply whether the training loss decreases, but whether the resulting representation preserves the information that prediction and planning require.

We will build and test these ideas using rendered observations from the DeepMind Control Suite (DMC)[1]. The experiments use tasks such as balancing a cart-pole and spinning a finger-controlled body. For the self-supervised experiments, the encoder receives images without the simulator's physics-state labels. We then use the simulator's state variables to evaluate what the learned representations retain. A separate supervised reference provides a comparison later in the chapter. These tests let us examine the representations directly instead of assuming that a successful training objective must have learned the right state.

> **TIP: Hardware, GPU selection, and rendering** The runners call `worldmodels.train.get_device()`, which selects CUDA when `torch.cuda.is_available()` is true and CPU otherwise. No device flag is needed,

> and the code does not select Apple MPS. The snippet below prints the selected device and, for CUDA, the GPU name. If it prints `cpu`, check that PyTorch has a CUDA-enabled build and that the driver exposes the GPU. Rendering through MuJoCo/OpenGL is separate; cached observations avoid repeating it.

```
import torch
from worldmodels.train import get_device
device = get_device()
print(f"Selected device: {device}")
print(f"CUDA available: {torch.cuda.is_available()}")
if device.type == "cuda":
    print(torch.cuda.get_device_name(device))
```

> For the supported Python version, environment creation, experiment commands, output-path cautions, and headless rendering help, follow `ch03/README.md`. On headless Linux, the DMC renderer may require `MUJOCO_GL=osmesa` and OSMesa system libraries. On Debian/Ubuntu, the design calls for `sudo apt-get install libgl1-mesa-glx libosmesa6`; check the repository README for platform-specific notes.

Unless stated otherwise, each number is the mean ± standard deviation over the actual trained seeds recorded for that experiment. The collapse demonstration and Table 3.3 use three seeds; the expanded SimSiam/BYOL/Barlow Twins study and primary CPC retrieval use five. Standard deviations are across trained model seeds, not independent episodes or trajectories.

We begin with simple pretext tasks, which reveal both the appeal and the limitations of deriving supervision from data. We then develop contrastive learning step by step, from constructing pairs of views to deriving the loss function. Next, we examine representation collapse and the different mechanisms used to avoid it. Masked prediction offers a different route: rather than comparing views, the model reconstructs parts of its input that were hidden from it. We then investigate how the size of the latent representation affects what can be recovered from it, before moving from individual observations to temporal prediction.

The chapter ends by assembling a reusable visual encoder. The static encoder maps a frame, or a fixed stack of frames, to a latent representation; it does not carry state from one call to the next. An optional recurrent wrapper can carry a hidden state across observations. Neither component alone is a complete world model: the latter also needs a learned transition model that predicts how the state changes under actions, along with components for evaluating or planning over possible futures. Chapter 4 develops predictive representations and action-conditioning further. For now, we study how learning signals shape representations and test what they retain.

> **NOTE: Three components, three different jobs**

> A **static visual encoder** maps the current image, or a fixed stack of images, to a latent vector z_t. It has no memory across calls. A **recurrent wrapper** is optional: it takes the current visual latent and a previous hidden state h_{t-1}, then returns an updated hidden state h_t. That hidden state is a learned model state, not the simulator's physical state. A **full world model** additionally needs a transition model that predicts future state or observations conditioned on actions, and components that evaluate or plan over possible futures. The GRU wrapper and CPC experiment in this chapter do not, by themselves, implement that full system.

Getting started: running the chapter code

The repository requires Python 3.11 or newer. From the repository root, create an environment, install the pinned dependencies, run the regression tests, and start with the single-seed contrastive experiment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python ch03/04_train_contrastive.py --seed 0 --checkpoints-dir checkpoints/quickstart
--figures-dir ch03/figures/quickstart --results-dir ch03/results/quickstart
```

On macOS or Linux, activate the environment with `source .venv/bin/activate` instead. The full multi-seed comparison, latent-dimension sweep, and CPC sequence experiment can take hours on CPU, depending on hardware and configuration. This chapter does not claim a universal runtime or measured GPU speed-up. The device selection applies to PyTorch training only; rendering the Control Suite is a separate dependency, and cached observations mean rendering is normally a one-time data-collection step.

The book's listings are short excerpts from the canonical `worldmodels/` implementations; the `ch03/` runners add the full data protocol, seed management, evaluation, plotting, and saved results. Start with the Chapter 3 code map, which documents experiment-specific settings and commands. Full multi-seed comparisons, the latent-dimension sweep, and CPC can take hours on CPU, depending on hardware and configuration. The runners select CUDA automatically when PyTorch detects it; the TIP above shows how to verify the selected device. Data collection is a separate rendering step, and downstream experiments read cached `.npz` files. When changing or reproducing an experiment, use isolated output paths so that canonical result JSONs, checkpoints, and figures are not overwritten.

3.1 Learning a state without labels

A useful state is not simply a smaller copy of an image. It is a description that preserves the

information needed to understand the situation. For a moving system, that may include positions and velocities, even when those quantities are not directly available as labels.

Let x denote an observation, such as an image rendered by a simulator. An encoder f_θ, parameterized by θ, transforms it into a representation z:

$$\boxed{z = f_\theta(x)}$$

We will use z for the representation throughout this chapter. It is a vector of d numbers, where d is the latent dimension. The encoder's job is to turn a high-dimensional observation into a form that later components of the world model can use.

But what should those numbers preserve? Chapter 2 gave us the answer at a conceptual level. A useful state must retain information that matters for predicting the future and discard information that does not. We can make that requirement concrete by separating two properties.

Sufficiency means that the representation preserves the information about the physical state that is available in the observation. If the representation loses a distinction that matters, a downstream dynamics model cannot recover it just by training harder.

Minimality means that the representation avoids carrying unnecessary information. For a cart-pole, this might mean retaining the cart's position and the pole's angle while ignoring irrelevant differences in brightness or background texture.

The two requirements must be considered together. An encoder that retains every pixel may preserve useful information, but it has not necessarily learned a compact state. An encoder that discards almost everything may be compact, but it can also lose the information needed to distinguish different situations.

3.1.1 Aliasing: when compression loses the wrong information

Suppose two cart-pole observations look identical. In one underlying situation, the cart is moving to the left; in the other, it is moving to the right. If both observations map to the same representation z, a downstream model receiving only that representation cannot distinguish the two situations.

This is an example of aliasing: different underlying states become indistinguishable in the representation.

Before examining individual objectives, Figure 3.1 maps their major families by the type of training signal. This overview updates the 2024 taxonomy supplied as a visual reference. The branches are not mutually exclusive: CPC is both temporal and contrastive, and masked prediction includes pixel, token, and latent targets. The methods later in the chapter are evaluated through their own measured tasks rather than ranked by this diagram.

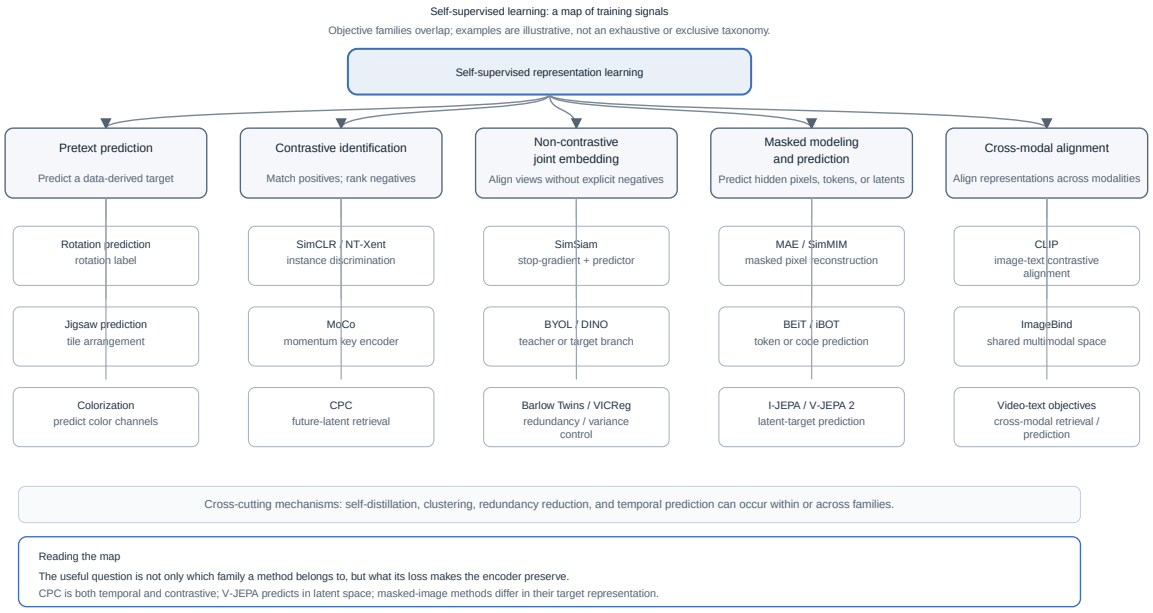


Figure 3.1 Self-supervised learning: a map of training signals

The categories are a teaching-oriented synthesis, informed by Gui et al. (2024), Uelwer et al. (2025), and Meta's V-JEPA 2 release (2025). The diagram is not an exhaustive or mutually exclusive taxonomy.

Table 3.1 Notation used in this chapter

Symbol	Meaning	Use in this chapter
x	Rendered observation or frame	Static visual input
y	Simulator physics-state target	Evaluation only for self-supervised methods; training target for the separately labelled supervised reference
f_θ	Static visual encoder with parameters θ	Maps pixels to features
z	Latent representation	Output of the visual encoder
d	Latent dimension	Capacity sweep in §3.6
t	Time index	Temporal sequences
z_t	Visual latent at time t	CPC inputs and retrieval candidates
c_t	CPC context vector	Summary of the encoded past for future-latent scoring
h_t	Optional recurrent-wrapper hidden state	Interface state; not simulator physics state

Symbol	Meaning	Use in this chapter
N	Source-frame count in a contrastive batch	Two views produce $2N$ embeddings
K	Number of negatives per anchor	Standard NT-Xent convention: $K = 2N - 2$
τ	Contrastive temperature	Controls softmax sharpness
M	Mask indicator for a patch	MAE loss is applied on masked patches
W_k	Horizon-specific CPC compatibility matrix	Scores latent at horizon k against context
R^2	Held-out state-probe score	Linear decodability measure, not a complete measure of information

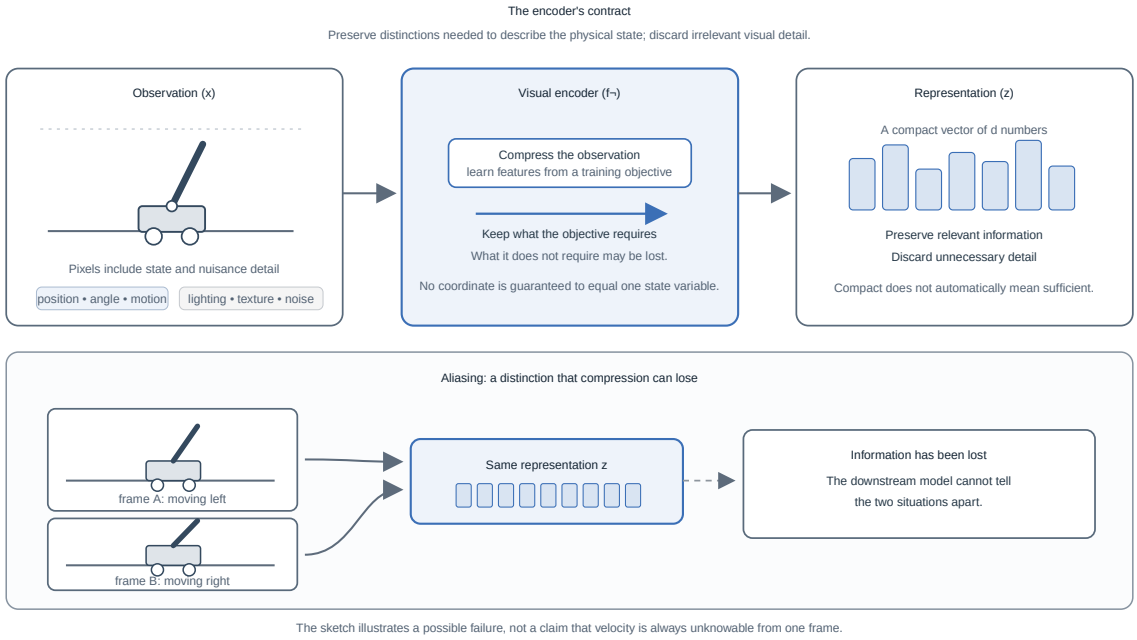


Figure 3.2 The encoder's contract

The example illustrates a possibility, not a universal claim that velocity is impossible to infer from a single image. A single frame may contain visual or statistical clues correlated with velocity. Whether those clues are enough is a question to measure, not an assumption to make. Later in the chapter, we will compare state measurements from single-frame representations with those from stacked frames.

3.1.2 Why not train directly on the state?

Let y denote the physical quantities we want the representation to retain. In our experiments, y consists of the simulator's generalized positions followed by its generalized velocities.

If y were available for every training observation, we could train an encoder to predict these quantities directly. The prediction error would provide a clear signal about which information the encoder needs to retain. In real settings, however, the exact physical state is often unavailable.

There are three difficulties with relying on labelled data alone.

Cost. Human annotations and physical measurements require effort or additional instrumentation. An agent can collect images continuously, but obtaining an exact label for every image may be expensive.

Task dependence. Labels reflect the task for which they were designed. An encoder trained to recognize objects might learn their identities while neglecting motion information needed by a dynamics model.

Availability. A deployed agent does not automatically know the exact positions and velocities of every object it observes. It receives images and whatever additional sensor readings are available to it.

Labels remain useful whenever they can be obtained. A simulator can supply its internal state, a synthetic data generator can produce targets, and demonstrations can provide training signals. These are legitimate alternatives, each with its own requirements. Self-supervised learning addresses a different opportunity: the observations themselves contain relationships that can be turned into learning signals without manually labelling every example.

> **DEFINITION — Self-supervised learning**

Self-supervised learning trains a model using a target constructed from the data itself, rather than a human-provided label. The model solves a task designed to produce this target, often called a pretext task. The pretext task is a means of learning a representation; its answer is not necessarily the final goal.

Consider a simple example. Take an image, rotate it by a known amount, and ask a model to identify the rotation. We do not need a person to annotate the transformed image: we already know which rotation we applied. The transformation generates the target automatically.

This clarifies the distinction between three learning settings. In supervised learning, an externally supplied label tells the model what to predict. In classical unsupervised learning, an objective seeks structure in the inputs without using an externally supplied label or a separately constructed prediction target. In self-supervised learning, the model has a target, but the data or a transformation of the data supplies it.

The terminology varies across the literature. Some surveys treat self-supervised learning as a subset of unsupervised learning; other authors emphasize that self-supervised methods do use a supervisory signal, even though it is generated from the data. This chapter uses the more explicit distinction above. For a broader taxonomy of self-supervised methods, see Gui and colleagues' A Survey on Self-supervised Learning: Algorithms, Applications, and Future Trends (2024).

> **NOTE — Predicting what is missing**

In his ICLR 2020 keynote, Yann LeCun described self-supervision through the general idea of predicting missing information (LeCun, 2020). The missing part might be a region of an image, the future given the past, or an obscured portion of an observation. These are not the same objective, but each turns structure already present in the data into a training signal.

This idea provides a useful map of the chapter: first learn from transformations and views, then from missing visual content, and finally from temporal relationships.

3.1.3 The objective determines what the encoder is rewarded for keeping

Self-supervised learning is not one algorithm. It is a family of methods that construct different training signals. Each signal rewards the encoder for retaining some information, while leaving other information optional.

Rotation prediction asks the model to identify a transformation applied to an image. It rewards features useful for recognizing that transformation. Those features need not be the features most useful for predicting a system's future.

Contrastive learning creates two altered views of the same observation and trains their representations to identify one another. The choice of alterations matters: features that survive those transformations can help identify a positive pair, while features that vary may be treated as less useful.

Masked prediction hides part of an observation and asks the model to predict the missing content from what remains visible. It rewards information that helps reconstruction. Some of that information may describe physical state; some may describe visual details irrelevant to control.

Temporal prediction asks the model to use earlier observations to identify or predict later ones. It provides a learning signal tied to what happens over time, although success still depends on the data, the prediction task, and the model.

The central principle is therefore:

> SIDEBAR — The principle behind this chapter**

A representation-learning objective specifies what information the representation is rewarded for retaining. Information the objective does not require may be discarded.

Contrastive learning rewards information that helps distinguish a positive view from other views. Masked prediction rewards information that helps reconstruct hidden content. Temporal prediction rewards information that helps identify or anticipate what comes next.

The objective is not the only influence. Architecture, training data, transformations, and training budget also shape the representation. A decreasing training loss does not, by itself, establish that the learned state is useful for prediction or planning.

This principle gives us a way to evaluate methods rather than simply collect them. We can examine

what their objectives encourage, then measure what their learned representations actually retain.

3.1.4 Why use a benchmark rather than a toy world?

The experiments in this chapter use the DeepMind Control Suite (DMC), a collection of physics-based control tasks built on MuJoCo. The observations are rendered by the simulator rather than drawn by a custom world designed solely to demonstrate a particular learning method. The suite gives us both rendered images for training and simulator variables against which to evaluate the resulting representations.

Our experiments use three tasks, shown in Figure 3.3.

- **Cartpole balance.** A pole is hinged to a cart that moves along a track. Its evaluation state has four variables: two generalized positions and two generalized velocities.
- **Finger spin.** A planar, two-joint finger interacts with a free body that can rotate. Its evaluation state has six variables: three generalized positions and three generalized velocities.
- **Cheetah run.** A planar, legged body moves across a surface. Its evaluation state has eighteen variables: nine generalized positions and nine generalized velocities.

Cartpole balance is the main task because its small state makes results relatively easy to interpret. Finger spin provides a second task for the objective comparison in Section 3.5. Cheetah run is used in Section 3.6 to examine how the size of the representation affects a task with a higher-dimensional state.

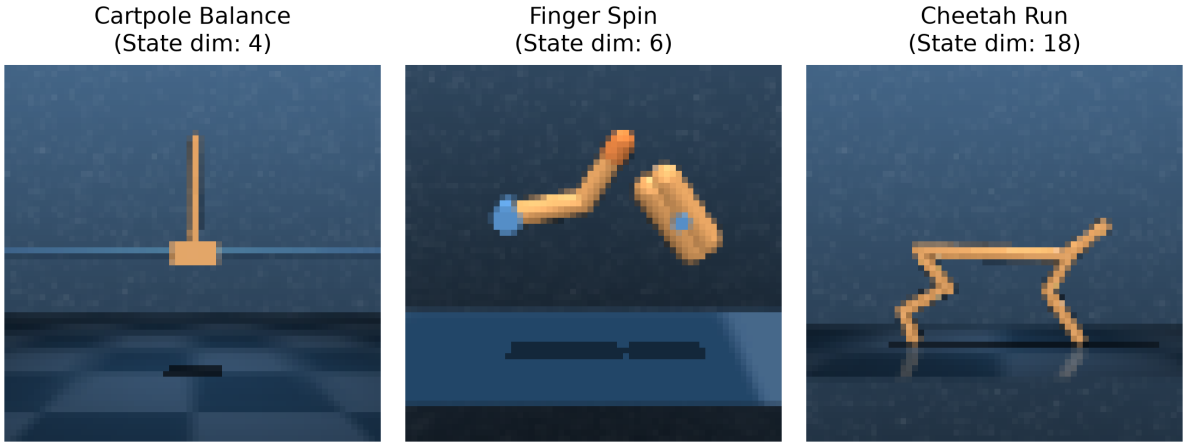


Figure 3.3 Rendered observations from the three tasks

3.1.5 The experimental contract

The data-collection code records rendered frames and the corresponding simulator state. It samples actions uniformly within each task's action bounds, steps the environment, and caches the observations and state variables. The training and validation datasets are collected using independent environment seeds.

The key distinction is between training signal and evaluation target. For the self-supervised

experiments, the training objectives receive rendered frames, not the stored physics-state labels. We use those labels to evaluate which physical quantities can be recovered from a learned representation. Section 3.5 also includes a separate supervised reference that is trained using state labels; that model is explicitly treated as a supervised comparison, not as a self-supervised method.

> **NOTE — Why the simulator state is still useful**

Using state variables for evaluation does not make the self-supervised objective supervised. It gives us a reference against which to test the representation after learning. The distinction must remain explicit: the self-supervised encoder does not receive these labels as its training target, while the supervised reference later in the chapter does.

Source: worldmodels/data/dmc_data.py. The excerpt below is taken from the existing data-collection implementation; the full episode loop and dataset-caching code remain in the repository.

Listing 3.1 Collecting observations and retaining the evaluation state

```
while not episode_done and frame_count < num_frames:
    pixels = env.physics.render(
        height=image_size[0],
        width=image_size[1],
        camera_id=camera_id,
    )
    state = _qpos_qvel_state(env.physics)

    action = rng.uniform(
        action_spec.minimum,
        action_spec.maximum,
        size=action_spec.shape,
    ).astype(np.float32)

    time_step = env.step(action)
    reward = float(time_step.reward or 0.0)
    episode_done = time_step.last()

    frames.append(pixels)
    physics_states.append(state)  # Evaluation target; not an SSL loss target
    actions.append(action)
    rewards.append(reward)
    dones.append(episode_done)
    episode_ids.append(current_episode)
    frame_count += 1
```

The helper _qpos_qvel_state in the same file returns the concatenation of MuJoCo's generalized positions $qpos$ and generalized velocities $qvel$. The module documentation explicitly states that these cached targets are used only for held-out evaluation and never enter self-supervised representation

training. The full data collection and reproducibility protocol lives in the runnable chapter script, not in this excerpt.

The purpose of this setup is not to assume that a particular representation will succeed. It gives us a consistent way to ask what each learning objective has preserved. Where a result matches our expectations, we can quantify the match. Where it does not, we will report the measurement at the scope supported by the experiment.

The next section begins with simple pretext tasks. They show how a target can be generated directly from an observation, and why solving a pretext task does not automatically guarantee that an encoder has learned the right state.

3.2 Pretext tasks: supervision from structure

A pretext task turns an unlabeled observation into a training example with a known target. The target comes from a transformation or relationship in the data, so we do not need a person to annotate each example.

Rotation prediction is a simple place to begin. It lets us construct the target explicitly, derive the loss from an ordinary classification problem, and then ask the central question of this chapter: does solving the pretext task require the encoder to preserve the information a world model needs?

3.2.1 Rotation prediction

Take a rendered cart-pole frame x. Rotate it by a multiple of 90 degrees and ask a network to predict which rotation was applied. We use four possible rotations: zero, 90, 180, and 270 degrees.

Let $r \in \{0, 1, 2, 3\}$ be the rotation label, where r counts quarter turns, and let $\rho_r(x)$ denote the corresponding rotated image. Starting with one frame gives us four training examples:

$$(\rho_0(x), 0), \quad (\rho_1(x), 1), \quad (\rho_2(x), 2), \quad (\rho_3(x), 3).$$

The label is known because we chose the transformation. No human annotation is needed.

Our frames are square, so rotating by a quarter turn simply rearranges the pixels. We do not need to introduce padding or interpolate new pixel values. The target is therefore determined by the applied rotation, rather than by an artifact created at the image border.

Now we need a network that predicts the rotation. We already have an encoder f_θ from equation 3.1. Attach a classification head g_ϕ, with parameters ϕ, to map the representation to four scores:

$$a(x') = g_\phi(f_\theta(x')).$$

Here x' is a rotated input, and $a_j(x')$ is the score, or logit, assigned to rotation class j. A logit is not yet a probability. We convert the four scores into probabilities using softmax:

$$\frac{\exp(a_j(x'))}{\sum_{m=0}^{3}\exp(a_m(x'))}.$$

The four probabilities are nonnegative and sum to one. The model's task is to assign a high probability to the rotation that was actually applied.

How should we measure whether it does so? Suppose the correct label is r. Its predicted probability is $p_{\theta,\phi}(r \mid x')$. We penalize the network using the negative logarithm of that probability:

$$-\log p_{\theta,\phi}(r \mid x').$$

This is the familiar cross-entropy loss for a classification problem with one correct class. If the model assigns probability one to the correct answer, the loss is zero. If it assigns a very small probability to the correct answer, the loss is large.

We average this loss over the four rotations of each original frame, then over the training frames. The result is the rotation-prediction objective:

$$-\mathbb{E}_x\left[\frac{1}{4}\sum_{r=0}^{3}\log p_{\theta,\phi}(r \mid \rho_r(x))\right]$$

The expectation averages over the training observations. The parameters θ and ϕ are learned together: the encoder must produce features that help the classification head identify the rotation.

A useful reference point is uniform prediction. If the model assigns probability $1/4$ to every rotation, the loss for each example is

$$-\log\frac{1}{4} = \log 4 \approx 1.386.$$

The value is measured in nats, because we use the natural logarithm. A model that predicts the correct class confidently can achieve a lower loss.

Notice what we have done. We have not supplied the physics state y, the cart position, or the pole angle as a training label. The objective only asks the model to identify the applied rotation. Equation 3.2 is an ordinary four-class classification loss; the special feature is how its labels are generated.

3.2.2 Two other pretext tasks

Rotation prediction is one example of using structure in the input to create a target. Two other classic tasks take a different approach.

Jigsaw prediction divides an image into a grid of tiles, shuffles them using a permutation selected from a predefined set, and trains the model to identify that permutation. The spatial relationships between tiles supply the learning signal. This can encourage features that capture local appearance and spatial arrangement, but it does not directly require the model to preserve physical variables such as velocity.

Colorization takes a grayscale image and trains a model to predict its color channels. The target is derived from the original image, so human labeling is unnecessary. Predicting color can encourage the model to use contextual information, but good color reconstruction is not the same as learning a state suitable for control. The objective rewards information useful for recovering color; it does not explicitly require every geometric or dynamical distinction that a downstream world model may need.

Figure 3.4 puts the three tasks side by side using the same cart-pole frame. Each task creates its own target, but the targets ask for different things.



Figure 3.4 artwork pending: Three pretext tasks from one frame

Production placeholder, not final artwork. See `../images/CH03_F04_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

Credit: Redraw and adapt the task overview from Figure 4 of Gui et al., A Survey on Self-supervised Learning: Algorithms, Applications, and Future Trends (2024), using our own Control Suite render for the example image. Cite the survey in the figure credit line.

NOTE

Gui and colleagues survey these and many other pretext tasks. This chapter uses rotation prediction because its target and loss are easy to derive. The purpose is not to cover every pretext task, but to understand what a chosen target requires the encoder to learn.

3.2.3 Why a correct pretext answer may still be the wrong goal

Return to the principle from Section 3.1: the training objective determines what information the representation is required to keep. Rotation prediction rewards features that help identify the rotation label. If the scene layout or background already makes that label easy to infer, the objective may be solvable without preserving the pole angle with the precision that control requires.

The same distinction applies to the other tasks. Jigsaw prediction rewards information useful for identifying tile arrangement. Colorization rewards information useful for predicting colors. Each objective can encourage useful features, and a trained encoder may retain more information than strictly necessary to solve its pretext task. But none of these losses, by itself, guarantees that the representation is sufficient for predicting the physical future.

That is the limitation we need to solve. Rather than ask the encoder to predict one fixed transformation, we want a learning signal that encourages it to ignore irrelevant changes while preserving the distinctions that matter. Small shifts or brightness changes might be nuisance variation; different physical situations should not be made indistinguishable merely because their images look similar.

This leads to instance discrimination, the starting point for contrastive learning. We will make two altered views of an observation and ask the model to recognize them as views of the same source. The choice of alterations will matter: it defines which changes the objective encourages the encoder to ignore. In the next section, we will build this idea from first principles and derive the loss that

implements it.

3.3 Contrastive learning from first principles

Section 3.2 showed how to manufacture a training target by transforming an observation. But the target determines what the encoder is asked to learn, and recognizing a rotation does not necessarily require a useful physical state.

Contrastive learning changes the question. Instead of asking which transformation was applied, it asks which observations came from the same source. We take two altered views of one frame and train a model to identify them as a pair among other views.

The idea is simple. Making two views agree encourages the encoder to retain information shared by both. Making it distinguish them from other views encourages the encoder to retain information that identifies the source. The mathematics turns this idea into a classification loss.

3.3.1 Views, positives, and negatives

A view of an observation x is an altered copy of it. Write the view as

$$\tilde{x} = t(x),$$

where t is a randomly selected transformation. The transformation is applied to the image, not to the underlying physical system.

In this chapter, the view pipeline uses two kinds of alteration. It shifts the image by a random integer offset of up to three pixels in each direction, then applies random brightness and contrast changes. The brightness offset is sampled from $[-0.1, 0.1]$, and the contrast factor from $[0.9,1.1]$. The values are clipped to the valid image range.

These changes are deliberately modest. They alter pixel values and image alignment, but they are intended to leave the physical situation unchanged.

A positive pair consists of two independently transformed views of the same frame. A negative pair consists of views from different source frames. The distinction comes from how we created the data, so no human label is needed.

The training process operates on a batch of N source frames. Each source frame produces two views, giving $2N$ images. The encoder maps these images to representations $z_1, \ldots, z_{2N}$.

For a particular view i, call it the anchor. Its positive, written i^+, is the other view of the same original frame. The remaining views are its negatives. Excluding the anchor itself, there are $2N-1$ candidates: one positive and $2N-2$ negatives.

Thus,

$$K = 2N - 2,$$

where K is the number of negatives for each anchor. For example, a batch of $N = 3$ frames produces six views. Each anchor has one positive and four negatives.



Figure 3.5 artwork pending: Constructing positives and negatives in a batch

Production placeholder, not final artwork. See `../images/CH03_F05_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

The transformation t is a design decision, not a cosmetic detail. If the encoder must identify two views of the same frame, changes introduced randomly by t cannot reliably identify the source. The encoder is therefore encouraged to become insensitive to those changes.

This is the intended benefit of the augmentation pipeline. If brightness changes do not alter the physical state, the representation should ideally remain useful despite them. But the same reasoning also creates a risk: if the transformation changes something important, the objective may reward the encoder for ignoring it.

A shift of a few pixels may be harmless for a cart-pole. A shift large enough to move the cart or pole out of view could erase information needed to estimate the state. The positive-pair definition therefore specifies which changes should be treated as irrelevant. The following equations show how the loss enforces this requirement.

The excerpt is from worldmodels/data/augment.py. It uses the existing ViewPipeline implementation.

Listing 3.2 Generating two views from a batch

```
def augment_single(self, images):
    x = random_crop_shift(
        images, max_shift=self.max_shift
    )                                                #A
    x = random_intensity_jitter(
        x,
        brightness_range=self.brightness_range,
        contrast_range=self.contrast_range,
    )                                                #B
    return x

def __call__(self, images):
    view_1 = self.augment_single(images)              #C
    view_2 = self.augment_single(images)
    return view_1, view_2
```

A Shifts each image by a randomly selected offset.

B Applies brightness and contrast jitter.

C Calls the pipeline twice, producing independent views.

The view pipeline is small, but its consequences run through the entire method. Contrastive learning can encourage invariance only to transformations that the training process actually applies. The choice of transformations is therefore part of the learning objective, even though it does not appear as a term in the loss equation.

> **NOTE — False negatives in trajectory batches**

A batch drawn from trajectories can contain frames that are almost identical, such as neighbouring steps from one episode. The basic objective still treats views from different source frames as negatives. These are called false negatives: two observations may differ in source identity while sharing much of their physical state. These numbers describe a hypothetical uniform draw from the full 30,000-frame cache, assuming 1,000 frames per episode: the probability that another frame comes from the same episode is $999/29,999$, about 3.3%, and the chance of selecting a frame within five steps is at most $10/29,999$, about 0.033%. The primary contrastive run actually samples a 5,000-frame episode-stratified subset, so these percentages are illustrative for the full cache; the experiment does not report an observed false-negative frequency for its sampled pool. We keep the simple in-batch rule and report its results rather than silently changing the negative set.

3.3.2 From similarity to a loss

We now need a numerical measure of how well two representations match. We will build the objective in three steps: define a similarity score, turn the candidate views into a classification problem, and minimize the classification error.

Step 1: Measure similarity

We use cosine similarity. For two nonzero vectors u and v, it is the cosine of the angle between them:

$$(u, v) = \frac{u^\top v}{\| u \|_2 \, \| v \|_2}.$$

Cosine similarity lies between -1 and 1. A value near 1 means the vectors point in similar directions; a value near -1 means they point in opposite directions. A value near zero means they are approximately orthogonal.

The normalization in the denominator means that the score depends on direction rather than vector length. Simply multiplying a representation by a positive constant cannot increase its cosine similarity with another vector.

Step 2: Turn similarity into a classification problem

Fix an anchor i. There are $2N-1$ candidate views, excluding the anchor itself. We want a classifier to assign the highest probability to the positive i^+.

For each candidate j, divide the similarity by a positive temperature τ, then apply softmax across the candidates:

$$p_i(j) = \frac{\exp\big((z_i, z_j)/\tau\big)}{\sum_{k \neq i} \exp\Big((z_i, z_k)/\tau\Big)}, \qquad j \neq i.$$

Here, $p_i(j)$ is the probability assigned to candidate j being the other view of the anchor's source frame. The correct answer is i^+.

The temperature τ controls how sharply differences in similarity affect the probabilities. A smaller temperature magnifies differences between candidates. A larger temperature makes their probabilities more uniform.

Step 3: Penalize a wrong answer

We use the negative logarithm of the probability assigned to the positive. The loss for anchor i is

$$\ell_i = -\log p_i(i^+).$$

A correct answer assigned high probability receives a small loss. A positive assigned low probability receives a large loss. Averaging across all $2N$ anchors gives the normalized temperature-scaled cross-entropy loss, or NT-Xent:

$$\frac{1}{2N} \sum_{i=1}^{2N} -\log p_i(i^+)$$

This is the same basic cross-entropy as in equation 3.2, but the classification problem is different. Rotation prediction chooses among four transformations. NT-Xent chooses among the other views in the batch, with a different correct class for each anchor.

Consider a batch with $N = 2$, so there are four views. Each anchor has three candidate views. Suppose an anchor has cosine similarities of 0.80 with its positive and 0.10 and -0.30 with the two negatives. Let the temperature be $\tau = 0.5$.

Dividing the similarities by the temperature gives logits of 1.6, 0.2, and -0.6. Their exponentials are approximately 4.953, 1.221, and 0.549. The sum is 6.723, so the softmax assigns probabilities of approximately 0.737, 0.182, and 0.082.

The positive has probability 0.737, giving the anchor a loss of

$$-\log(0.737) \approx 0.306.$$

If the classifier cannot distinguish the positive and assigns the same probability to all three candidates, it assigns probability $1/3$ to the positive and has loss

$$-\log(1/3) = \log 3 \approx 1.099.$$

For general batch size N, an exactly uniform probability distribution over the candidates gives loss $\log(2N-1)$. This is the uninformative reference value for this classification problem.

> **WARNING — Comparing NT-Xent losses**

Compare NT-Xent loss values only when the batch size and temperature are the same. The number of candidate views changes with batch size, and the temperature changes the softmax probabilities. A lower numerical loss under a different configuration does not necessarily mean a better representation.

> **TIP — Batch size controls the negatives**

Batch size sets the number of negatives: $K = 2N - 2$. A larger batch asks the model to distinguish the positive from more candidates, but it also requires memory for more views and their similarity scores.



Figure 3.6 artwork pending: NT-Xent for one anchor

Production placeholder, not final artwork. See `../images/CH03_F06_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

3.3.3 What InfoNCE actually optimizes

NT-Xent is one instance of the broader InfoNCE objective, used in contrastive predictive coding (van den Oord et al., 2018) and closely related to the contrastive formulation used by SimCLR (Chen et al., 2020). For each anchor, InfoNCE compares one positive with K negatives. Its mathematical properties help explain both what contrastive learning encourages and what it cannot guarantee.

First, we need a measure of shared information.

> **DEFINITION — Mutual information**

The mutual information $I(a; b)$ between random variables a and b measures how much knowing one reduces uncertainty about the other. It is the expected log ratio between their joint distribution and the product of their marginal distributions:

$$I(a; b) = \mathbb{E}_{p(a,\ b)}\left[\log \frac{p(a,\ b)}{p(a)p(b)}\right].$$

Mutual information is zero when a and b are independent. With natural logarithms, it is measured in nats.

Let a and b be the representations of two views of the same frame. This is the positive pair. Let $b_1, \ldots, b_K$ be representations of negative views drawn independently from the marginal distribution of b. For a positive score function (f$a,\ b$), InfoNCE is

$$-\mathbb{E}\left[\log \frac{f(a,\ b)}{f(a,\ b) + \sum_{k\ =\ 1}^{K} f(a,\ b_k)}\right].$$

For NT-Xent, the score uses exponentiated cosine similarity divided by temperature. Under the standard sampling assumptions, InfoNCE satisfies the lower bound

$$\boxed{I(a;\, b) \geq \log(K+1) - \mathcal{L}_{\text{InfoNCE}}}$$

This is a result about the information shared by the two views' representations, not a direct guarantee that the representation captures the physical state.

Proof note. The mutual-information lower bound follows from the Bayes-optimal classifier over one positive and K marginal negatives. Appendix A gives the complete change-of-measure argument; here we use the bound only as an interpretation of shared view information, not as a guarantee that physical state is retained.

Equation 3.6 has two useful implications. First, lowering the InfoNCE loss raises the conventional lower bound on the information shared by the positive pair. Second, the bound is capped by $\log(K+1)$, because the loss cannot be negative. A finite batch therefore limits how much shared information this objective can certify.

The assumptions matter. In the chapter's implementation, the negatives come from other frames in a batch, and two views from the same source frame are themselves related. Frames drawn from trajectories may also be correlated. The textbook mutual-information bound is therefore a useful interpretation of the objective, not an exact measurement of the trained model's mutual information.

The deeper limitation follows from how the views are constructed. Each view is a randomly transformed copy of the same source frame x. If z_1 and z_2 are their representations, the data-processing inequality gives

$$I(z_1;\, z_2) \leq I(\tilde{x}_1;\, \tilde{x}_2) \leq I(x;\, \tilde{x}_1).$$

The information shared by both views is constrained by what survives the transformations. A detail that changes independently between views cannot reliably identify a positive pair. Conversely, information preserved across both views can help.

But shared information is not the same as the information needed by a world model. The objective asks the encoder to distinguish one frame's views from other frames' views. It does not explicitly ask it to preserve every state variable useful for prediction or planning.

The gradient in equation 3.7 separates attraction toward the positive from repulsion by negatives. The coefficient of each negative is its current softmax probability, so hard negatives receive more weight. Appendix A derives the gradient and the cosine-similarity derivative step by step.

$$\boxed{\begin{aligned}\nabla_{z_i} \ell_i = &-\frac{1 - p_i(i^+)}{\tau} \nabla_{z_i}(z_i,\, z_{i^+}) \\ &+ \sum_{k \notin i,\, i^+} \frac{p_i(k)}{\tau} \nabla_{z_i}(z_i,\, z_k).\end{aligned}}$$

The first term attracts the anchor toward its positive. The second term pushes it away from the negatives, weighted by their current probabilities.

Four observations follow.

Balanced forces: the sum of the negative probabilities is $1 - p_i(i^+)$, so the total negative weight equals the attraction weight for this anchor.

Self-limiting updates: when the positive already receives probability close to one, both attraction and repulsion become small.

Hard negatives matter most: a negative with high similarity receives a larger probability and a stronger push.

Temperature changes the emphasis: lowering τ makes the probability distribution more concentrated, increasing the relative influence of the most competitive negative.

In the numerical example above, the attraction weight is $(1 - 0.737) / 0.5 \approx 0.527$. The two negative weights are $0.182 / 0.5 = 0.364$ and $0.082 / 0.5 = 0.164$. Their sum is approximately equal to the attraction weight, with the small discrepancy due to rounding. Gradient descent applies the update in the opposite direction of the gradient, increasing similarity to the positive and decreasing similarity to negatives.

equation 3.7 describes the anchor's role in one loss. The same representation also serves as a candidate in the losses of other anchors; those additional terms contribute to its complete gradient.

The key limitation remains: these gradients arrange the representations to solve the specified identification problem. The objective does not know whether the features used for identification are cart position, pole angle, texture, or some combination. That is why the following experiments evaluate the encoder rather than treating a falling loss as proof of a useful state.

3.3.4 Training a contrastive encoder

We now train the loss on rendered frames from cartpole_balance. The model has two parts: a convolutional encoder that produces the representation used later, and a projection head used during contrastive training.

The encoder in worldmodels/models/encoders.py contains four convolutional layers with 4×4 kernels and stride two. Starting with 64 × 64 RGB input, the spatial resolution is reduced as follows:

$$64 \to 32 \to 16 \to 8 \to 4.$$

The channel count grows from 32 to 64, 128, and 256. The final 256 × 4 × 4 feature map is flattened and projected to a 16-dimensional vector $z = f_\theta(x)$.

Source: worldmodels/models/encoders.py. This excerpt is condensed from the canonical implementation, with the architecture and behavior unchanged.

Listing 3.3 The convolutional encoder and projection head

```
class ConvEncoder(nn.Module):
    def __init__(
        self,
        in_channels: int = 3,
        latent_dim: int = 16,
        base_channels: int = 32,
    ) -> None:
        super().__init__()
        self.in_channels = in_channels
        self.latent_dim = latent_dim
        self.conv_net = nn.Sequential(
            nn.Conv2d(in_channels, base_channels, 4, 2, 1),
            nn.ReLU(inplace=True),
            nn.Conv2d(base_channels, base_channels * 2, 4, 2, 1),
            nn.ReLU(inplace=True),
            nn.Conv2d(base_channels * 2, base_channels * 4, 4, 2, 1),
            nn.ReLU(inplace=True),
            nn.Conv2d(base_channels * 4, base_channels * 8, 4, 2, 1),
            nn.ReLU(inplace=True),
        )
        flat_dim = (base_channels * 8) * 4 * 4
        self.fc = nn.Linear(flat_dim, latent_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if x.ndim != 4:
            raise ValueError("ConvEncoder expects input shaped (B, C, H, W)")
        features = self.conv_net(x)
        return self.fc(torch.flatten(features, start_dim=1))

class ProjectionHead(nn.Module):
    def __init__(self, in_dim: int = 16, hidden_dim: int = 64, out_dim: int = 16) ->
None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_dim, out_dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

A Each convolution halves the spatial height and width.

B Maps the flattened feature map to the 16-dimensional representation.

C Maps the representation to the projection used by the contrastive loss.

With the default channel count, the encoder has 755,696 parameters and the projection head has 2,128. The encoder uses no batch normalization, so the representation of one frame does not depend on the other images in its batch.

The loss is applied to the projection $p = g_\phi(z)$, not directly to z. This follows the design used in SimCLR, where a nonlinear projection head is used during contrastive pretraining while the representation before that head is retained for downstream tasks (Chen et al., 2020). The reason for separating the two is practical: the projection can become invariant to some details needed by later tasks, while the encoder representation remains available to downstream components. This chapter adopts that setup without claiming that the two alternatives have been compared experimentally here.

Source: worldmodels/losses/ntxent.py, the canonical implementation.

Listing 3.4 The NT-Xent loss

```
def nt_xent_loss(z1, z2, temperature=0.5):
    n = z1.shape[0]
    device = z1.device
    z1 = F.normalize(z1, dim=1)                          #A
    z2 = F.normalize(z2, dim=1)
    reps = torch.cat([z1, z2], dim=0)                   #B
    logits = torch.matmul(reps, reps.T) / temperature #C
    eye = torch.eye(2 * n, dtype=torch.bool, device=device)
    logits = logits.masked_fill(eye, -1e9)              #D
    labels = torch.cat([
        torch.arange(n, 2 * n, device=device),           #E
        torch.arange(0, n, device=device),
    ])
    return F.cross_entropy(logits, labels)               #F
```

A Normalizes each representation so its dot product becomes cosine similarity.

B Concatenates the two views into one batch of $2N$ vectors.

C Computes all pairwise similarities, scaled by the temperature.

D Removes self-similarity from each anchor's candidate set.

E Identifies the positive row for each view.

F Applies cross-entropy to the positive labels, implementing equation 3.5.

Masking the diagonal excludes the anchor itself from the softmax denominator. The label indices identify the other view of the same frame: the positive of row i is row $i + N$, and the positive of row $i + N$ is row i.

The repository also includes ch03/03_ntxent_from_scratch.py. It compares a direct loop-based implementation of the mathematical definition with the vectorized function above, checking both loss values and gradients. It is a verification script, rather than a second implementation intended for the main training run.

The training loop

The training script ch03/04_train_contrastive.py loads cached cart-pole frames, creates the two views, runs both through the shared encoder, applies the projection head, and evaluates NT-Xent. The optimizer updates the encoder and projection-head parameters together.

Source: ch03/04_train_contrastive.py, condensed from the existing runner.

Listing 3.5 One contrastive training step

```
for batch in train_loader:
    images = batch["image"].to(device)
    view_1, view_2 = view_pipeline(images)              #A
    z1 = encoder(view_1)
    z2 = encoder(view_2)
    p1 = proj_head(z1)
    p2 = proj_head(z2)                                  #B
    loss = nt_xent_loss(
        p1, p2, temperature=args.temperature
    )                                                   #C
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()                                    #D
```

A Creates two independently transformed views of every frame.

B Encodes each view and applies the projection head.

C Computes NT-Xent on the two batches of projected features.

D Backpropagates the loss and updates both networks.

The recorded run uses 5,000 training frames sampled evenly across the 30 training episodes, a batch size of 64, a 16-dimensional latent, Adam with learning rate 10^{-3}, temperature $\tau = 0.5$, and 15 epochs. The held-out validation set is sampled across the five validation episodes. The training frames are selected across episodes and across time within each episode; the script does not simply take the first 5,000 frames.

A batch contains $N = 64$ source frames, or 128 views, giving $K = 126$ negatives per anchor. Results are recorded for three seeds: 0, 1, and 2. The physics state is stored with each frame, but the contrastive training objective operates on the images and projected representations, not on that state.

Figure 3.7 shows the recorded three-seed NT-Xent training loss on Cartpole. The mean falls from approximately 3.779 in the first epoch to 3.106 in the final epoch, with a seed standard deviation of about 0.005 at epoch 15. The curve is still decreasing at the final epoch, so this is the end of the chosen training budget, not a demonstrated convergence point.

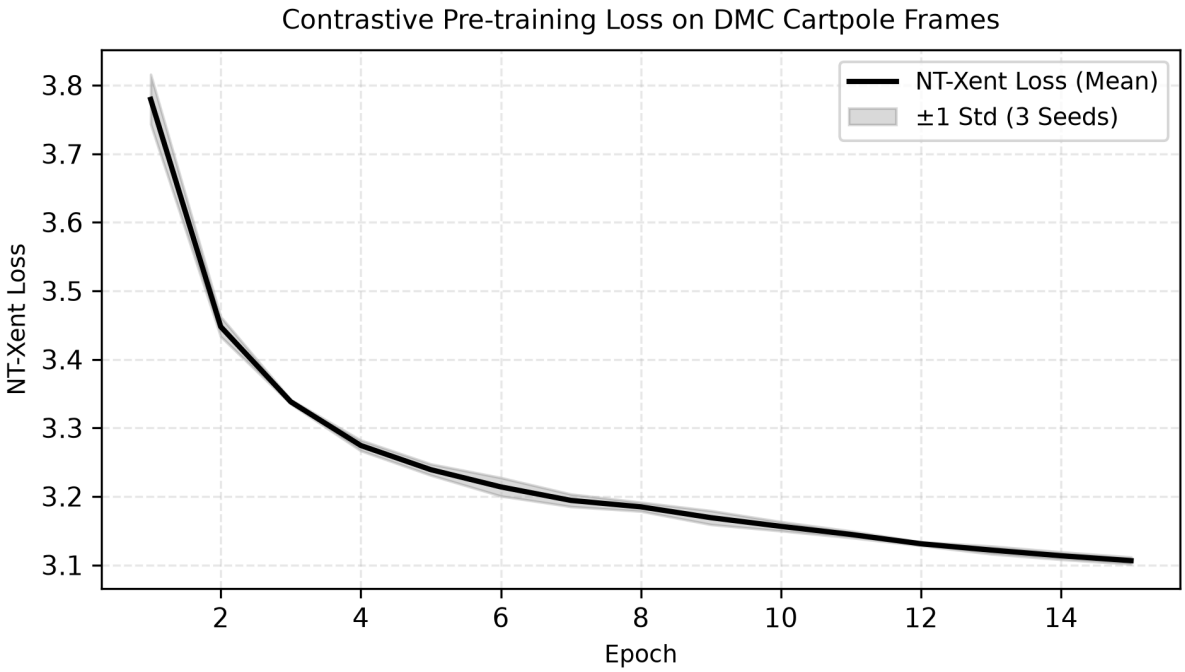


Figure 3.7 NT-Xent training loss on Cartpole

The loss is still decreasing at the final epoch, so this run ends at its chosen training budget rather than at a demonstrated convergence point. Its trend is evidence that the network is learning to solve the batch-identification task. It is not, by itself, evidence that the representation has captured the physical state.

Figure 3.8 is a qualitative two-component PCA projection of validation representations. The production plot must use wrapped pole-angle bins and episode-stratified validation sampling, as specified in `images/CH03_F08_Dadwal.md`. A two-dimensional projection cannot establish which physical variables are linearly recoverable.



Figure 3.8 artwork pending: PCA projections of validation representations

Production placeholder, not final artwork. See `../images/CH03_F08_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

The visual projection can show that the numerical outputs change during training, but it cannot tell us what the dimensions encode. We now need an evaluation that asks the physical question directly.

3.3.5 Reading the state back with a linear probe

The simulator provides the physical state associated with each rendered frame. That state never enters the self-supervised loss. We can use it after training to ask which physical quantities are available in

the representation.

A linear probe is a simple evaluation model fitted to a frozen encoder. We compute the representation z, freeze the encoder, and fit a linear map from z to the evaluation targets. We then evaluate that map on held-out observations. This measures the information that a linear readout can recover; it does not test whether an arbitrary nonlinear decoder could recover more.

For cart-pole balance, the raw state contains four variables: cart position, pole angle, cart velocity, and pole angular velocity. The pole angle needs special treatment. The simulator stores its hinge angle as an unwrapped value, so angles separated by a full revolution can have different numbers but represent the same physical orientation. An angle of 0.1 radians and one of $0.1 + 2\pi$ radians produce the same orientation.

To make a linear probe appropriate for this variable, the evaluation targets replace the angle θ with $\sin\theta$ and $\cos\theta$. The transformed target therefore has five components: cart position, $\sin\theta$, $\cos\theta$, cart velocity, and pole angular velocity. The probe reconstructs an angle using the inverse tangent, and the repository scores angle predictions with a circular R^2-like metric. This avoids treating a prediction across the angular wrap boundary as a large error simply because the numerical angle differs by nearly 2π.

Let $\check{z}_n$ be a representation standardized using the training-set feature means and standard deviations, and let $\tilde{y}_n$ be the transformed target vector. The probe fits a ridge-regression readout:

$$\min_{W,\, b} \sum_{n=1}^{M} \| \tilde{y}_n - W\check{z}_n - b \|_2^2 + \alpha \| W \|_F^2$$

Here, M is the number of training examples used to fit the probe; W and b are the linear weights and bias; and α controls the ridge penalty. The current probe implementation uses $\alpha = 10^{-3}$, standardizes the frozen features, and solves the ridge problem by least squares. The encoder is not updated when the probe is fitted.

Source: worldmodels/eval/probes.py, the canonical LinearProbe implementation.

Listing 3.6 Fitting a ridge probe to frozen features

```
def fit(self, x, y):
    x_scaled = self._fit_feature_scaler(x)            #A
    n, d = x_scaled.shape
    x_bias = np.concatenate(
        [x_scaled, np.ones((n, 1))], axis=1
    )                                                 #B
    reg = np.zeros((d + 1, d + 1))
    reg[:d, :d] = np.sqrt(self.alpha) * np.eye(d)     #C
    augmented_x = np.vstack([x_bias, reg])
    augmented_y = np.vstack([
        y, np.zeros((d + 1, y.shape[1]))
```

```
    ])
    self.weights, *_ = np.linalg.lstsq(
        augmented_x, augmented_y, rcond=None
    )                                                    #D
    return self
```

A Standardizes features with statistics from the probe's training data.

B Adds a bias column, which is not penalized.

C Adds the rows that implement the ridge penalty.

D Solves the augmented least-squares problem corresponding to equation 3.8.

The score is computed on held-out validation frames. For ordinary state variables, $R^2 = 1$ is perfect prediction, $R^2 = 0$ matches the constant-mean baseline, and a negative value is worse than that baseline. Angular targets use the circular score described above.

A probe's limitations are important. A high score means the physical variable is available to a linear readout. A low or negative score does not prove the information is absent from z; it may be represented nonlinearly, or the probe may be limited by the data and fitting setup. The measurement tells us what is explicitly and linearly available under this evaluation, not everything the encoder may know.

Does adding a second frame recover velocity?

The aliasing diagnostic compares a contrastive encoder receiving a single frame with one receiving two consecutive frames stacked along the channel dimension. Both are evaluated using a frozen-feature linear probe. The experiment uses three seeds, a latent dimension of 16, 12 training epochs, up to 2,000 training examples, and up to 500 validation examples. For the two-frame case, selected frame windows do not cross episode boundaries.

Table 3.2 State probes from one frame and a two-frame stack

State variable	Single frame R^2	Two-frame stack R^2	Change
Cart position	0.714 ± 0.040	0.707 ± 0.010	-0.007
Pole angle	0.612 ± 0.069	0.586 ± 0.064	-0.026
Cart velocity	0.006 ± 0.004	0.009 ± 0.007	$+0.002$
Pole angular velocity	-0.367 ± 0.001	-0.375 ± 0.006	-0.007

All values are mean ± standard deviation across three seeds. The table uses the recorded results in ch03/results/table_03_03_aliasing_diagnostic.json; angular state is scored with the circular metric.

The result is more limited than the hypothesis that a second frame will recover velocity. Position is more readily decoded than velocity from a single frame, but stacking two frames does not reliably

improve the velocity measurements in this configuration. Cart-velocity R^2 increases by only about 0.002, while the pole-angular-velocity score becomes slightly worse. The position scores also do not improve.

This is a result about the tested image resolution, view transformations, encoder, data sampling, and training budget. It does not prove that velocity cannot be inferred from image sequences. It shows that simply providing two consecutive frames was not enough to make the tested velocities reliably available to this linear probe.

The main contrastive training run and this aliasing diagnostic also use different training budgets, so their results should not be treated as a controlled comparison with all other factors held fixed. In both cases, the probes use state labels only for evaluation. The contrastive objective itself still sees rendered frames and generated view pairs.

The larger lesson is the one that motivated this chapter. The encoder is optimized for the task stated by its loss. NT-Xent rewards representations that distinguish a frame's augmented views from views of other frames. That can organize representations and make source identity more accessible, but the loss does not explicitly require the physical state needed by a world model. To learn that state, the objective and the information available to the encoder matter.

The next section examines a failure that can occur even before asking which physical variables are represented: collapse, in which different inputs map to nearly identical representations and the model satisfies a similarity objective without learning a useful distinction.

3.4 The collapse problem and how it is avoided

3.4.1 Demonstrating collapse

Consider the simplest possible objective for two views of each frame: make their representations equal. Let $p_n^{(1)}$ and $p_n^{(2)}$ denote the vectors compared by the loss for the two views of frame n. For a batch of N frames and representations of dimension d, define

$$\frac{1}{Nd}\sum_{n=1}^{N} \| p_n^{(1)} - p_n^{(2)} \|_2^2.$$

This objective is nonnegative. It reaches zero whenever the two views produce identical representations.

But it does not require different frames to produce different representations. Suppose the encoder returns the same constant vector for every input. Then every positive pair matches exactly, the loss is zero, and the objective has reached its global minimum. The solution works regardless of what the images contain. It does not need to represent cart position, pole angle, or any other aspect of the physical state.

This is complete collapse. All inputs map to one point.

There is also dimensional collapse. Here, the representation still varies across inputs, but its variation is restricted to only a few of the available dimensions. The encoder may produce a 32-dimensional vector while using only a small number of independent directions to distinguish observations. These are different failure modes. We therefore need more than one measurement to diagnose them.

Measuring collapse

The first measurement is spread: the standard deviation of each coordinate across a collection of representations, averaged over all d coordinates. If the encoder returns a constant vector for every input, the spread is zero. If its outputs become almost identical, the spread approaches zero.

Spread alone is not enough, however. A representation can use many independent directions while all its coordinates have very small values. Conversely, a representation can have large coordinate variation but vary mainly along one direction.

The second measurement is effective rank, which summarizes how many directions carry substantial variation.

Let $\lambda_1, \ldots, \lambda_d$ be the eigenvalues of the representation covariance matrix, and define the normalized eigenvalues

$$q_j = \frac{\lambda_j}{\sum_{\ell=1}^{d} \lambda_\ell}.$$

The effective rank is the exponential of the entropy of this distribution:

$$\exp\left(-\sum_{j=1}^{d} q_j \ln q_j\right).$$

If the variance is distributed equally across all d directions, the effective rank is d. If all variation lies in one direction, the effective rank is one. Intermediate values indicate that variation is distributed across a smaller number of directions.

The normalization removes overall scale. Multiplying every representation by 10^{-5} does not change the effective rank, because all covariance eigenvalues are multiplied by the same squared factor and the normalized q_j remain unchanged. This is useful for measuring dimensional collapse, but it means effective rank cannot detect a representation whose spread has shrunk almost to zero.

The following excerpt is from the diagnostic code in ch03/06_collapse_demo.py. It computes both measurements from a matrix of representations.

Listing 3.7 Measuring spread and effective rank

```
def feature_diagnostics(features):
    features = np.asarray(features, dtype=np.float64)
    centered = features - features.mean(axis=0, keepdims=True)
    singular_values = np.linalg.svd(centered, compute_uv=False)
```

```
    eigenvalues = singular_values ** 2                  #A
    total = float(eigenvalues.sum())
    if total <= 1e-15:
        rank = 0.0
    else:
        q = eigenvalues / total                          #B
        entropy = -np.sum(q * np.log(np.maximum(q, 1e-15)))
        rank = float(np.exp(entropy))                    #C
    spread = float(np.std(features, axis=0).mean())      #D
    return {"mean_dim_std": spread, "effective_rank": rank}
```

A The squared singular values give the covariance eigenvalues up to a common factor.

B Normalizes the eigenvalues, removing the influence of overall scale.

C Computes the effective rank from the entropy.

D Averages the standard deviation across representation coordinates.

The full repository function also validates its input and handles the degenerate all-zero case. We use this diagnostic on the encoder output, not just the projection head, because the encoder output is the representation retained for later use.

What happens when we remove the negatives?

The repository's collapse demonstration compares two training conditions using the same convolutional encoder, projection head, data, optimizer, and view pipeline. The only intended difference is the objective: one run uses mean-squared attraction between positive views; the other uses NT-Xent.

The experiment uses cart-pole balance, 3,000 training frames sampled across episodes, batch size 64, 15 epochs, and three seeds. It measures the spread of the encoder output before training and after every epoch. The recorded results are stored in ch03/results/collapse_demo_metrics.json.



Figure 3.9 artwork pending: Encoder spread with and without negatives

Production placeholder, not final artwork. See `../images/CH03_F09_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

Under attraction only, the encoder spread falls from approximately 5.2×10^{-4} at initialization to 7.8×10^{-5} after 15 epochs. The training loss falls to approximately 2.6×10^{-10}. Under NT-Xent, the encoder spread reaches approximately 2.42, while the final training loss is approximately 3.16. The two conditions behave very differently: attraction alone can achieve an almost-zero loss while allowing its already small representation variation to shrink further.

This finding needs a careful interpretation. The randomly initialized encoder already has a very small spread in this experiment. The run therefore does not demonstrate that a useful, well-trained

representation was destroyed by attraction-only training. It demonstrates that the attraction loss does not require diversity and allows the representation to remain near-constant.

Effective rank gives a complementary result. Under attraction only, the final encoder effective rank remains about 12.4 out of 16, even as spread shrinks to roughly 10^{-5}. Under NT-Xent, effective rank drops sharply early in training and then recovers to about 7.0 by the final epoch. An effective-rank measurement alone would therefore have failed to identify the attraction-only representation's extremely small scale.

> **NOTE — Interpreting small spread**

A small spread is not, on its own, proof that a representation contains no information. A representation can encode distinctions at a small scale. The recorded experiment does not probe the attraction-only encoder for physical-state information, so we do not claim that its tiny variation is necessarily useless. The result is narrower: the objective does not demand spread or prevent a constant representation from minimizing the loss.

Why does NT-Xent behave differently? If all views map to the same nonzero vector, every cosine similarity is one. The positive and the negatives then have identical scores, so the probability assigned to the positive is $1/(2N-1)$ and the loss is $\ln(2N-1)$. The encoder cannot reduce the loss by distinguishing the positive from the negatives, because it has collapsed them all together. NT-Xent rewards a lower loss when the positive receives a higher score than the negatives.

The negative pairs therefore give contrastive learning a mechanism that attraction alone lacks. But they bring their own design choices, including the number and quality of the negatives. The next section examines alternatives that do not require them.

3.4.2 Three mechanisms in brief

SimSiam, BYOL, and Barlow Twins use different mechanisms to avoid collapse without contrastive negative pairs. Their shared starting point is that two views of the same frame should produce related representations. What differs is how each method prevents that requirement from being satisfied by an uninformative constant output.

The repository implements reduced-scale versions of all three methods. The study uses cart-pole frames, five random seeds, 30 epochs, up to 6,000 training frames, batch size 128, and Adam with learning rate 10^{-3}. The encoder output has 32 dimensions, while the projection and predictor heads use 128-dimensional embeddings. These experiments examine the core mechanisms at small scale; they are not exact reproductions of the large-scale ImageNet experiments in the original papers.

SimSiam: stop the gradient through the target

SimSiam uses a shared encoder and projector for the two views, plus a predictor applied to one branch. Let z_1, z_2 be the projected representations and p_1, p_2 their predictor outputs.

For a predictor output p and target representation z, define the negative cosine similarity:

$$D(p, z) = -\frac{p}{\| p \|_2} \cdot \frac{z}{\| z \|_2}.$$

The symmetric objective evaluates each view as the prediction and the other view as the target:

$$\frac{12}{D}\Big(p_1, (z_2)\Big) + \frac{12}{D}\Big(p_2, (z_1)\Big)$$

The operator [stem b612eeef4ccf6658aa1eb7c15510a140] means stop-gradient. It leaves the target's value unchanged, but treats it as a constant when calculating the gradient. The predictor is trained to match the opposite branch's representation, while the target branch does not receive gradients through that particular loss term.

This asymmetry is central to SimSiam. The method also uses a projector and a predictor with batch normalization, following its original design. The stop-gradient changes the optimization process, but it is not a universal mathematical guarantee that every possible configuration avoids collapse. Its effectiveness is established primarily through the method's experiments, and our reduced-scale study measures the result of the full configuration rather than performing an ablation of each component.

BYOL: a slowly moving target

BYOL also compares online predictions with target representations, but its target branch is a separate network. The online network contains an encoder, projector, and predictor and is updated by gradient descent. The target network contains an encoder and projector, receives no gradient, and changes through an exponential moving average of the online parameters.

Let θ_o denote the online parameters, θ_t the target parameters, and $m \in [0, 1)$ the momentum. The update is

$$\boxed{\theta_t \leftarrow m\theta_t + (1-m)\theta_o}$$

The target therefore changes gradually rather than tracking every online update immediately. The online network learns to predict the slowly changing target. In the repository implementation, the momentum starts at 0.99 and increases toward one during training using a cosine schedule.

The target network is not optimized by gradient descent. Its parameters are updated by the exponential moving average, which provides the asymmetry in BYOL's learning procedure. As with SimSiam, this mechanism is not a general proof of collapse prevention for every configuration. The experiment tests the implemented method at the declared scale and budget.

Source: worldmodels/losses/noncontrastive.py.

Listing 3.8 Stop-gradient and the target-network update

```
def simsiam_loss(p1, z2, p2, z1):
    def d(p, z):
        p = F.normalize(p, dim=1)
```

```
        z = F.normalize(z.detach(), dim=1)              #A
        return -(p * z).sum(dim=1).mean()                #B

    return 0.5 * d(p1, z2) + 0.5 * d(p2, z1)            #C


def update_target_ema(online_model, target_model, decay=0.99):
    with torch.no_grad():
        for po, pt in zip(
            online_model.parameters(), target_model.parameters()
        ):
            pt.data.mul_(decay)                          #D
            pt.data.add_(po.data, alpha=1.0 - decay)
```

A Detaches the target from the gradient computation.

B Computes negative cosine similarity for the batch.

C Averages the two directions, as in equation 3.10.

D Moves the target parameters toward the online parameters, as in equation 3.11.

The code above contains two distinct mechanisms. SimSiam stops the gradient through the target representation; BYOL updates the target network by exponential moving average. They should not be conflated: BYOL's target-network update is not simply another spelling of SimSiam's stop-gradient.

Barlow Twins: reduce redundancy between coordinates

Barlow Twins takes a different approach. It standardizes each coordinate of the two embedding batches and computes their cross-correlation matrix. If the views agree coordinate by coordinate, the diagonal should approach one. If different coordinates are uncorrelated, the off-diagonal entries should approach zero.

Let $\hat{z}^{A}_{b,\,i}$ and $\hat{z}^{B}_{b,\,j}$ denote standardized coordinates i and j for frame b in two view batches. The cross-correlation matrix is

$$C_{ij} = \frac{1}{N}\sum_{b=1}^{N} \hat{z}^{A}_{b,\,i}\hat{z}^{B}_{b,\,j}.$$

The loss penalizes deviation of diagonal entries from one and penalizes nonzero off-diagonal entries:

$$\sum_{i}(1 - C_{ii})^2 + \lambda\sum_{i}\sum_{j \neq i} C_{ij}^2$$

The diagonal term rewards agreement across views. The off-diagonal term discourages different dimensions from carrying redundant, linearly correlated information. The implementation uses $\lambda = 0.005$.

A constant coordinate cannot be standardized into a meaningful unit-variance signal. The code clamps the standard deviation at 10^{-6}; if a coordinate is constant, its standardized values are zero, and its diagonal term contributes a penalty of one. Collapse therefore incurs a positive loss. However, because the method standardizes the features, this objective does not set a minimum scale for the unstandardized representation itself.

Source: worldmodels/losses/noncontrastive.py.

Listing 3.9 The Barlow Twins cross-correlation loss

```
def barlow_twins_loss(z1, z2, lambd=0.005):
    n, d = z1.shape
    s1 = z1.std(dim=0, unbiased=False).clamp_min(1e-6)
    s2 = z2.std(dim=0, unbiased=False).clamp_min(1e-6)
    a = (z1 - z1.mean(dim=0)) / s1                      #A
    b = (z2 - z2.mean(dim=0)) / s2
    c = a.T @ b / n                                     #B
    diag = (torch.diagonal(c) - 1).pow(2).sum()         #C
    off = ~torch.eye(d, dtype=torch.bool, device=z1.device)
    off_diag = c[off].pow(2).sum()                      #D
    return diag + lambd * off_diag
```

A Standardizes every coordinate across the batch.

B Computes the cross-correlation matrix in equation 3.12.

C Penalizes diagonal entries that differ from one.

D Penalizes off-diagonal correlations.



Figure 3.10 artwork pending: Three mechanisms for avoiding collapse

Production placeholder, not final artwork. See `../images/CH03_F10_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

What did the reduced-scale study measure?

The recorded comparison in ch03/results/three_mechanisms_diagnostics.json reports frozen-encoder state-probe R^2, effective rank, and feature spread for the three implementations. Across five seeds, the mean probe scores are 0.266 for SimSiam, 0.268 for BYOL, and 0.285 for Barlow Twins.

The effective ranks, out of 32 encoder dimensions, are 4.23 for SimSiam, 7.07 for BYOL, and 11.18 for Barlow Twins. Their average coordinate standard deviations differ markedly: approximately 1.93, 0.306, and 0.0616, respectively.

These measurements illustrate why spread, effective rank, and state probes must be read together. Barlow Twins has the highest mean probe score in this run, but its advantage is modest relative to

variation across seeds. Its much smaller spread does not, by itself, mean that its representation has collapsed: the Barlow Twins objective standardizes coordinates before applying the loss, and the frozen-feature probe also standardizes its inputs. The rank and probe provide complementary information.

The runs also do not establish a universal method ranking. They use a common reduced-scale study on cart-pole frames, but the architectures and training mechanisms differ, and the experiment is not an exact reproduction of the original large-scale benchmarks. Nor does the comparison isolate one loss term while holding every other architectural choice fixed. Its conclusion is limited to the measured configurations: all three methods retained nontrivial variation under these settings, with different representation statistics and similar mean probe scores.

The mechanisms examined so far prevent collapse through asymmetry or relationships between coordinates. VICReg makes the requirements explicit in three terms: maintain agreement between views, keep variation in each coordinate above a floor, and reduce covariance between different coordinates.

3.4.3 VICReg: making the anti-collapse force explicit

VICReg stands for variance-invariance-covariance regularization. It receives two batches of embeddings, Z^A and Z^B, each with N rows and d coordinates. It combines three terms, each with a distinct purpose.

The variance term

The variance term sets a floor on the standard deviation of each coordinate across a batch. Let $Z_{:,j}$ denote column j, and let S be its regularized standard deviation:

$$S(u) = \sqrt{(u) + \epsilon}.$$

The variance hinge is

$$\boxed{v(Z) = \frac{1}{d}\sum_{j=1}^{d} \max\left(0, \gamma - S(Z_{:,j})\right)}$$

Here, γ is the minimum target standard deviation and ϵ>0 stabilizes the square root. The repository uses $\gamma = 1$ and $\epsilon = 10^{-4}$.

The hinge is zero for a coordinate whose standard deviation is already at least γ. A coordinate below the threshold incurs a penalty proportional to the shortfall. The term therefore sets a floor; it does not reward arbitrarily increasing a coordinate's scale.

The small ϵ matters near zero variance. Without it, differentiating the square root at zero can produce an undefined or unbounded derivative. With $\epsilon = 10^{-4}$, the derivative of $\sqrt{v + \epsilon}$ with respect to variance is bounded above by $1/(2\sqrt{\epsilon}) = 50$.

The invariance term

The invariance term asks the two views to produce similar embeddings. In the implementation, it is the mean squared error between matching rows:

$$s(Z^A, Z^B) = \frac{1}{Nd} \sum_{n=1}^{N} \| z_n^A - z_n^B \|_2^2$$

This is the positive-view attraction from the collapse demonstration, but now it is only one part of the objective. By itself, it would allow a constant representation. In VICReg, the variance and covariance terms constrain what solutions can minimize the total loss.

The covariance term

The covariance term discourages redundancy between different coordinates. Let $\bar{z}$ be the mean row of the batch. Define the sample covariance matrix

$$C(Z) = \frac{1}{N-1} \sum_{n=1}^{N} (z_n - \bar{z})(z_n - \bar{z})^\top.$$

Its diagonal contains the coordinate variances. Its off-diagonal entries contain the covariances between different coordinates. The VICReg covariance term penalizes squared off-diagonal entries:

$$c(Z) = \frac{1}{d} \sum_{i \neq j} [C(Z)]_{ij}^2$$

If all off-diagonal entries are zero, the coordinates are pairwise uncorrelated in the batch. This reduces second-order redundancy, but it does not establish that the coordinates are statistically independent: nonlinear dependence can remain even when covariance is zero.

Combining the three terms

The total VICReg loss is a weighted sum

$$\begin{aligned} L_{\text{VICReg}} = \; & \lambda s(Z^A, Z^B) \\ & + \mu \big[v(Z^A) + v(Z^B) \big] \\ & + \nu \big[c(Z^A) + c(Z^B) \big]. \end{aligned}$$

The implementation uses $\lambda = 25$, $\mu = 25$, and $\nu = 1$. These coefficients set the relative influence of invariance, variance, and covariance penalties. Both embedding batches receive their own variance and covariance penalties.

Consider what the three terms do to degenerate solutions.

A constant representation. If every coordinate is constant across the batch, the invariance term is zero and the covariance term is zero. But the regularized standard deviation of each coordinate is only $\sqrt{\epsilon} = 0.01$. With $\gamma = 1$, each coordinate incurs a variance-hinge penalty of 0.99. Across both views, the weighted variance term adds $25 \times 2 \times 0.99 = 49.5$ to the total loss. Collapse is no longer a zero-cost solution.

A single repeated direction. Suppose all coordinates are identical copies of one unit-variance scalar. The views can match and every coordinate meets the variance floor, so the invariance and variance terms can both be zero. But every off-diagonal covariance equals one. With $d = 16$, there are $16 \times 15 = 240$ off-diagonal entries, and the covariance cost for each batch is $240 / 16 = 15$. Across the two views, the covariance term contributes 30. This discourages dimensional collapse into one repeated direction.

Unrelated but varied embeddings. A model could produce embeddings with sufficient variance and little coordinate covariance without making the two views of the same frame agree. The variance and covariance terms would not necessarily penalize this configuration, but the invariance term would. It is the term that ties the representations of paired views together.

These examples show why no one of the three terms replaces the others. Variance prevents coordinates from becoming constant, covariance discourages redundancy, and invariance links representations of positive views. Their combination makes the constraints explicit, instead of relying solely on the behavior of the training procedure.

Supplementary code excerpt — VICReg loss (not a numbered listing)

Source: worldmodels/losses/vicreg.py.

```
def vicreg_loss(za, zb, sim_coeff=25.0, std_coeff=25.0,
                cov_coeff=1.0, gamma=1.0, eps=1e-4):
    n, d = za.shape
    sim = F.mse_loss(za, zb)                              #A

    def variance(z):
        v = z.var(dim=0, unbiased=False)
        std = torch.sqrt(v + eps)                         #B
        return F.relu(gamma - std).mean()                 #C

    def covariance(z):
        z = z - z.mean(dim=0)
        cov = (z.T @ z) / (n - 1)                        #D
        off = ~torch.eye(d, dtype=torch.bool, device=z.device)
        return cov[off].pow(2).sum() / d                  #E

    var = variance(za) + variance(zb)
    cov = covariance(za) + covariance(zb)
    total = sim_coeff * sim + std_coeff * var + cov_coeff * cov
```

```
    return total, sim, var, cov
```

A Computes the invariance term from equation 3.15.

B Computes the regularized standard deviation used by equation 3.14.

C Penalizes coordinates whose standard deviation is below γ.

D Computes the batch covariance matrix.

E Sums the squared off-diagonal entries, divided by d.

The variance calculation uses population variance, while the covariance estimate divides by (N-1), matching the repository implementation. The function returns the total loss and each of its three components so the training script can record their behavior separately.

Training VICReg on cart-pole frames

The standalone VICReg runner is ch03/08_vicreg.py. It uses the convolutional encoder and projection head from Section 3.3.4, with 4,000 training frames, batch size 64, 15 epochs, Adam at learning rate 10^{-3}, and three seeds. The three loss terms operate on the 16-dimensional projection-head outputs; the downstream state probe reads the encoder output before the projection head.

The recorded VICReg curves in Figure 3.11 show the total loss and three unweighted loss components. The variance hinge falls close to zero by around epoch five, while the covariance term rises early and then declines. The invariance term remains comparatively small. This pattern is consistent with the variance floor being established before the covariance penalty reduces redundancy, though the experiment does not separately test that causal interpretation.



Figure 3.11 artwork pending: VICReg terms and their training curves

Production placeholder, not final artwork. See `../images/CH03_F11_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

In this standalone configuration, the state probe reports mean $R^2 = 0.248 \pm 0.005$ across three seeds. Cart position and pole angle are more linearly recoverable than the velocities. These results describe this run's architecture, sample cap, training budget, and probe. The contrastive training run in Section 3.3 uses a different budget and number of training frames, so these scores should not be interpreted as a controlled comparison of objectives. The objective-comparison study later in Section 3.5 uses a separately specified protocol.

VICReg avoids explicit negative pairs, but that does not make it free of design choices. The covariance matrix is estimated from a finite batch, and the coefficients λ, μ, ν determine the balance between the three terms. This chapter uses the stated coefficients rather than tuning them to obtain a preferred result.

The methods in this section address collapse through different mechanisms: NT-Xent uses negative

pairs, SimSiam and BYOL introduce asymmetric training paths, Barlow Twins reduces cross-correlation, and VICReg imposes explicit variance and covariance constraints. None of these mechanisms alone establishes that the resulting representation is a sufficient physical state. The next section explores a different learning signal: rather than aligning two representations, a masked autoencoder learns to reconstruct parts of the observation that were hidden from it.

3.5 Masked prediction

3.5.1 From masked words to masked patches

BERT trains a language model by hiding selected tokens and asking it to predict them from the surrounding context. The targets are the identities of the missing tokens, not a human-written label for the whole sentence. The model learns from the structure already present in the text.

A masked autoencoder MAE applies a related idea to images. It divides an image into patches, hides a random subset, and predicts the missing pixel values from the visible patches. The training target comes directly from the input image. No class label, action, reward, or physics state is required.

The masking ratio matters. If only a few patches are hidden, nearby visible pixels may make reconstruction relatively easy. Hiding a large fraction of the image creates a more demanding prediction task: the model must use the remaining patches and their relationships to reconstruct the missing content. The original MAE work found that a high masking ratio, such as 75%, could produce a useful self-supervised task at scale. That result motivates our choice; it does not establish that 75% is optimal for every dataset or architecture.

Our experiment uses 64×64 RGB frames from the Control Suite's Cartpole Balance task. Each image is divided into an 8×8 grid, giving 64 patches. With a masking ratio of 75%, the encoder sees 16 randomly selected patches during training. The decoder predicts all 64 patches, but only the masked patches contribute to the loss.

This is an asymmetric architecture. The encoder processes the visible patches without carrying a placeholder token for every hidden patch. The decoder receives the encoded visible patches, inserts learned mask tokens into the missing positions, restores the original order, and predicts the pixel values. This division keeps the encoder's training input smaller than the full image's patch sequence.

There is an important boundary to the analogy with language. A rendered physics frame is far simpler than a natural image, and our images contain a stable background. Reconstructing a Cartpole frame is not the same as reconstructing the physical state that generated it. In particular, a single image need not reveal velocity. The experiment tests what this compact MAE learns under its particular data, architecture, and training budget.

3.5.2 Building a masked autoencoder

The first operation is to turn an image into a sequence of patches. For a batch of images, the resulting tensor has shape

$$P \in \mathbb{R}^{B \times N \times D},$$

where B is the batch size, $N = 64$ is the number of patches, and $D = 8 \times 8 \times 3 = 192$ is the number of pixel-channel values in each flattened patch.

A learned linear layer embeds each patch into a 64-dimensional token. Positional embeddings distinguish the patches' locations. The model then samples a different random mask for each image in the batch. It retains 16 tokens and sends only those tokens through the encoder.

Source: worldmodels/models/masked_autoencoder.py; reconstruction loss: worldmodels/losses/reconstruction.py. This is a shortened excerpt; patchification and decoder helpers remain in the canonical module.

Listing 3.10 Masking and the MAE forward pass

```
def forward_encoder(self, x: torch.Tensor):
    patches = self.patchify(x)
    tokens = self.patch_embed(patches) + self.encoder_pos_embed
    b, n, d = tokens.shape
    len_keep = int(n * (1.0 - self.mask_ratio))

    noise = torch.rand(b, n, device=x.device)
    ids_shuffle = torch.argsort(noise, dim=1)
    ids_restore = torch.argsort(ids_shuffle, dim=1)
    ids_keep = ids_shuffle[:, :len_keep]
    visible = torch.gather(
        tokens, 1, ids_keep.unsqueeze(-1).expand(-1, -1, d)
    )

    mask = torch.ones(b, n, device=x.device)
    mask[:, :len_keep] = 0.0
    mask = torch.gather(mask, 1, ids_restore)

    encoded_visible = self.encoder_transformer(visible)
    z = encoded_visible.mean(dim=1)
    return z, encoded_visible, mask, ids_restore


def forward(self, x: torch.Tensor):
    patches = self.patchify(x)
    _, encoded_visible, mask, ids_restore = self.forward_encoder(x)
    pred_patches = self.forward_decoder(encoded_visible, ids_restore)
    loss = masked_mse_loss(patches, pred_patches, mask)
    return loss, pred_patches, mask
```

The random values in noise define a permutation of the patch positions. ids_keep selects the visible tokens, while ids_restore records how to put the visible tokens and mask tokens back into their original

order for decoding. The mask uses 1 for a hidden patch and 0 for a visible patch. These details are important: a decoder must know which predicted patch belongs in which image location.

The training objective is the mean squared error over masked patches only. Let P_{bi} denote patch i from image b, let $\hat{P}_{bi}$ be its prediction, and let M_{bi} equal 1 if that patch was masked and 0 otherwise. With D pixel-channel values per patch, the loss is

$$\frac{\sum_{b=1}^{B}\sum_{i=1}^{N} M_{bi}\left[\frac{1}{D}\sum_{j=1}^{D}(P_{bij}-\hat{P}_{bij})^2\right]}{\sum_{b=1}^{B}\sum_{i=1}^{N} M_{bi}}.$$

The denominator counts masked patches, so each hidden patch contributes its mean pixel-channel error. Visible patches do not contribute to the loss, even though the decoder produces predictions for them too. This matches the implementation in masked_mse_loss.

The physics states collected from the simulator do not enter this objective. They remain available for evaluation, including the linear-probe tests used elsewhere in the chapter. The only training target here is the hidden image content.



Figure 3.12 artwork pending: A masked autoencoder and its reconstructions

Production placeholder, not final artwork. See `../images/CH03_F12_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

The completed Figure 3.12 combines real reconstruction outputs with a separately drawn architecture schematic. The reconstruction panel alone does not show the encoder's visible-token path, mask-token restoration, or masked-patch loss; these remain part of the production brief in `images/CH03_F12_Dadwal.md`.

The MAE implementation exposes a 64-dimensional representation for evaluation. Specifically, SimpleMAE.encode() runs the encoder over all image patches and mean-pools its final tokens. The linear probe uses this representation directly. The constructor still accepts a legacy latent_dim argument, but that argument does not set the feature dimension used by the current MAE probe.

NOTE — Other generative objectives: Variational autoencoders and generative adversarial networks introduce additional objectives and trade-offs. They are deferred to the generative chapters.

3.5.3 Reconstruction versus representation

A reconstruction loss measures how well the model rebuilds its input. A state probe asks a different question: how much information about the simulator's state can be recovered by a linear map from the learned representation? Good reconstruction does not guarantee a strong state representation, just as a strong state probe does not guarantee pixel-accurate reconstruction.

Consider the target in equation 3.18. The error is measured across the pixels of every masked patch,

including patches containing the floor and background. These pixels are part of the image and therefore part of the objective, even when their appearance says little about the cart's position or the pole's angular velocity. The encoder can be rewarded for retaining information that helps reconstruct pixels without being equally rewarded for making every physical variable easy to decode linearly.

This is a hypothesis about the objective, not a conclusion that follows from the equation alone. We test it by examining reconstructions and measuring frozen-feature linear probes. The interpretation must remain bounded by the experiment: Cartpole frames at 64×64, the current patch-transformer architecture, a 75% masking ratio, and the stated training budget. These frames are far simpler than natural video, so this comparison does not establish a general ranking of reconstruction and contrastive methods.

The probe results

The standalone MAE experiment uses seeds 0, 1, and 2, 4,000 episode-stratified training frames, 1,000 validation frames, 15 epochs, a batch size of 64, and Adam with learning rate 10^{-3}. Its probe uses the mean-pooled 64-dimensional encoder tokens. Across the three training seeds, the mean R^2 across the four Cartpole state variables is 0.170 ± 0.004. This is the result recorded in ch03/results/mae_metrics.json.

Table 3.3 compares complete method configurations rather than loss functions in isolation. Each method uses seeds 0, 1, and 2, 12 epochs, a batch size of 64, learning rate 10^{-3}, 3,500 training samples, and 1,000 validation samples. Its position and velocity scores aggregate their respective state variables. The reported uncertainty is the standard deviation across the three trained seeds, not across independent environments.

Panel A. Aggregate position and velocity scores. Values are mean ± population SD across three trained seeds.

Table 3.3 Method-family state-probe comparison

Task	Encoder / method	Position R^2	Velocity R^2
Cartpole Balance	Supervised-pretrained encoder + frozen-feature linear probe	0.981 ± 0.000	−0.273 ± 0.012
Cartpole Balance	Contrastive (NT-Xent)	0.672 ± 0.001	−0.187 ± 0.005
Cartpole Balance	VICReg	0.649 ± 0.022	−0.183 ± 0.006
Cartpole Balance	Masked autoencoder (MAE)	0.475 ± 0.012	−0.156 ± 0.005
Finger Spin	Supervised-pretrained encoder + frozen-feature linear probe	0.995 ± 0.001	−0.114 ± 0.010
Finger Spin	Contrastive (NT-Xent)	0.714 ± 0.029	−0.007 ± 0.002

Task	Encoder / method	Position R^2	Velocity R^2
Finger Spin	VICReg	0.679 ± 0.040	−0.005 ± 0.004
Finger Spin	Masked autoencoder (MAE)	0.383 ± 0.010	−0.034 ± 0.001

Panel B. Per-variable state-probe scores.

Task	Method	Per-variable R^2 in state-vector order (Finger Spin qpos[2] uses π-periodic scoring)
Cartpole	Supervised reference	0.994 ± 0.000; 0.967 ± 0.000; −0.098 ± 0.016; −0.448 ± 0.008
Cartpole	Contrastive	0.735 ± 0.023; 0.608 ± 0.025; 0.002 ± 0.008; −0.376 ± 0.005
Cartpole	VICReg	0.782 ± 0.040; 0.516 ± 0.005; −0.006 ± 0.015; −0.361 ± 0.009
Cartpole	MAE	0.483 ± 0.025; 0.467 ± 0.001; −0.032 ± 0.009; −0.281 ± 0.014
Finger Spin	Supervised reference	0.997 ± 0.000; 0.993 ± 0.001; 0.999 ± 0.000 (π-periodic); −0.070 ± 0.008; −0.095 ± 0.002; −0.178 ± 0.021
Finger Spin	Contrastive	0.941 ± 0.017; 0.487 ± 0.045; 0.996 ± 0.001 (π-periodic); −0.010 ± 0.001; −0.008 ± 0.002; −0.003 ± 0.003
Finger Spin	VICReg	0.974 ± 0.006; 0.383 ± 0.074; 0.941 ± 0.003 (π-periodic); −0.013 ± 0.003; −0.011 ± 0.001; 0.008 ± 0.009
Finger Spin	MAE	0.895 ± 0.011; −0.128 ± 0.029; 0.939 ± 0.023 (π-periodic); −0.031 ± 0.005; −0.025 ± 0.006; −0.045 ± 0.010

For Finger Spin, the absolute spinner hinge angle is excluded from the aggregate position score. Its rendered geometry has π-periodic symmetry, so a static image cannot uniquely identify its absolute 2π angle. The position summary therefore averages the other position variables, and the velocity summary averages the task's velocity variables. The exclusion is part of the metric definition, not a discarded unsuccessful prediction.

The supervised row has a specific interpretation. The encoder and a prediction head were trained using privileged simulator-state labels, after which a separate linear probe was fit to frozen encoder features. The reported numbers belong to that separate probe, not to the trained prediction head's held-out score. The supervised reference is not a self-supervised method.

The per-variable results make the aggregates easier to interpret. The entries are mean R^2 ± standard deviation across the same three seeds and follow the state-vector order: Cartpole uses qpos[0], qpos[1], qvel[0], qvel[1]. Finger Spin uses qpos[0], qpos[1], qpos[2], qvel[0], qvel[1], qvel[2]. For Finger Spin qpos[2], the table reports a separate π-periodic score for visible orientation; absolute 2π angle remains unidentifiable from a static render. That score is displayed for completeness but excluded from the aggregate position score.

R^2 values below zero mean that the probe performs worse on the validation data than a constant predictor using the validation-target mean. In these runs, velocity is difficult to decode linearly across methods. The MAE's Cartpole position score is below those of the contrastive and VICReg representations in this comparison, while its Cartpole velocity score is less negative. On Finger Spin, MAE performs poorly on the second position variable and has negative velocity scores. No single aggregate tells the whole story.

These observations do not isolate the objective as the cause. MAE uses a patch-transformer encoder and decoder, and its evaluated feature vector is 64-dimensional. The contrastive and VICReg representations in this comparison use a convolutional backbone with a 16-dimensional latent representation. The supervised reference additionally uses privileged state labels during pretraining. Table 3.3 is therefore a method-family comparison under a stated budget, not a loss-only ablation. The measurements describe these complete configurations; they do not establish that masked reconstruction is inherently inferior to contrastive learning or VICReg.



Figure 3.13 artwork pending: Joint embedding and pixel reconstruction

Production placeholder, not final artwork. See `../images/CH03_F13_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

The result is consistent with the chapter's central distinction: the training objective specifies what the encoder is directly rewarded for preserving. Masked prediction is useful when reconstructing hidden observations is the desired task, but pixel reconstruction alone does not ensure that the resulting features expose the physical state. World models ultimately need a representation that can support prediction and planning, not just one that can rebuild an image.

Before predicting in a representation space, we should ask how much representational capacity the state needs. The next section studies that question through latent bottlenecks.

3.6 Latent bottlenecks and compact state

3.6.1 The information bottleneck as a lens

Let X denote an observation, Z its learned representation, and Y a task-relevant variable. For a world

model, X might be an image and Y might describe the physical state needed to predict what happens next.

The information bottleneck asks for a representation that preserves information relevant to Y while discarding as much unnecessary information about X as possible. One way to express this trade-off is

$$\mathcal{L}_{\text{IB}} = I(Z; X) - \beta I(Z; Y).$$

Here, $I(A; B)$ is the mutual information between random variables A and B. It measures statistical dependence: it is zero when the variables are independent and increases as knowing one variable provides more information about the other. The coefficient β controls the exchange between compression and relevance.

The two terms pull in opposite directions.

Compression. Minimizing $I(Z; X)$ discourages the representation from carrying information about every detail of the input.

Relevance. Minimizing the full loss rewards a large $I(Z; Y)$, because that term is subtracted. The representation should preserve information that helps distinguish task-relevant outcomes.

When β is small, compression receives more weight relative to relevance. As β increases, preserving information about Y becomes more important. The best trade-off depends on which information the task actually needs.

This gives a useful interpretation of a compact state. If an image contains a cart, a pole, a floor, lighting, and background details, the agent may not need to encode all those details equally. It needs enough information to distinguish states that matter for prediction and control. But compression must stop before two physically different situations that require different decisions become indistinguishable in the representation.

That is the connection to the contracts from Chapter 2. A representation needs sufficiency for the task, but minimality asks it not to retain unnecessary detail. A bottleneck can help with the second goal while making the first harder to satisfy.

There is an important distinction between this information-theoretic objective and simply choosing a small vector. A deterministic encoder with a low-dimensional continuous output is not, by that fact alone, optimizing $I(Z; X)$. In formal information-bottleneck formulations, the relationship between input and representation is treated probabilistically, and the mutual-information terms must be defined and optimized or bounded appropriately. A deterministic autoencoder with a narrow latent layer imposes a practical capacity constraint; it does not automatically implement equation 3.19.

The same distinction applies to VICReg. Its invariance, variance, and covariance penalties encourage particular properties of learned embeddings, but VICReg does not optimize the information-bottleneck objective in equation 3.19. The equation is a conceptual lens for understanding what we want from a representation, not a claim that every self-supervised objective is secretly an information-bottleneck algorithm.

In this chapter's experiments, the simulator's physics state remains an evaluation target. The contrastive loss does not receive that state as a training label, and the latent-dimension sweep does not calculate mutual information directly. We use a linear probe to measure one concrete property: how much of the state is linearly recoverable from the learned features.

3.6.2 Choosing the latent dimension

The practical question is how many dimensions the encoder needs. It is tempting to set the latent dimension equal to the number of physical state variables. That is not a reliable rule.

A physical variable need not map to one latent coordinate. A learned feature can combine several state variables, and a single variable may require several features to become easy to decode. This is especially relevant to angles: a linear probe can have difficulty reading an angle directly near the wraparound boundary, so the evaluation uses periodic encoding and a circular R^2 score for angular positions.

Nor does a larger latent dimension automatically make the representation better. The encoder may fail to learn useful features within the available training budget, or its additional coordinates may not improve the information that the probe can recover. The dimension is a capacity choice; the probe is a measurement of the result.

The hypothesis

The benchmark supplies two tasks with different physical-state dimensions:

Cartpole Balance: four state variables.

Cheetah Run: eighteen state variables.

The hypothesis is that Cartpole will reach a useful probe-performance plateau at a smaller latent dimension, while Cheetah will benefit from larger representations for longer. At $d = 2$, Cartpole's representation may be too restrictive to preserve the variation needed to distinguish its physical states.

This hypothesis concerns the performance of the complete learned representation, not just the arithmetic difference between the number of latent dimensions and the number of state variables. A four-variable system does not imply that a four-dimensional latent is sufficient, nor that four is the optimal dimension for a linear probe.

The experiment

The repository's ch03/10_bottleneck_sweep.py trains a separate contrastive encoder for every combination of task, latent dimension, and random seed. The dimensions are $d \in 2, 4, 8, 16, 64, 256$, with seeds 0, 1, and 2. Each run uses the convolutional encoder from Section 3.3, with a projection head whose output dimension matches d.

The run uses Adam with a learning rate of 10^{-3}, a batch size of 64, and 10 training epochs. It selects 3,000 training frames and 1,000 validation frames, spreading the selected indices across episodes

rather than taking a single prefix of each dataset. The contrastive objective operates on two augmented views of each frame. Once training is complete, the linear probe is fitted to the frozen encoder features, not the projection head's output.

Source: `ch03/10_bottleneck_sweep.py`. The full runner handles task configuration, seeds, episode-aware sampling, checkpointing, metrics, and plot generation.

Listing 3.11 The latent-dimension sweep driver

```
def train_and_evaluate(
    train_loader, val_loader, latent_dim, seed, args, device
):
    set_seed(seed)
    encoder = ConvEncoder(latent_dim=latent_dim).to(device)
    projector = ProjectionHead(latent_dim, 64, latent_dim).to(device)
    optimizer = torch.optim.Adam(
        list(encoder.parameters()) + list(projector.parameters()),
        lr=args.lr,
    )
    pipeline = ViewPipeline()

    for _ in range(args.epochs):
        encoder.train()
        projector.train()
        for batch in train_loader:
            x = batch["image"].to(device)
            v1, v2 = pipeline(x)
            loss = nt_xent_loss(
                projector(encoder(v1)),
                projector(encoder(v2)),
            )
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    return evaluate_linear_probe(
        encoder, train_loader, val_loader, device
    )
```

The sweep measures held-out linear-probe R^2, averaged across the task's physical-state variables. The probe standardizes frozen latent features before fitting a ridge-regression readout. Its angular-state evaluation uses periodic handling, so the score is not distorted merely because an angle crosses a wraparound boundary. The reported spread is the standard deviation across the three trained seeds, not across independently collected environments.

The measured result

The recorded sweep output is shown in Figure 3.14.

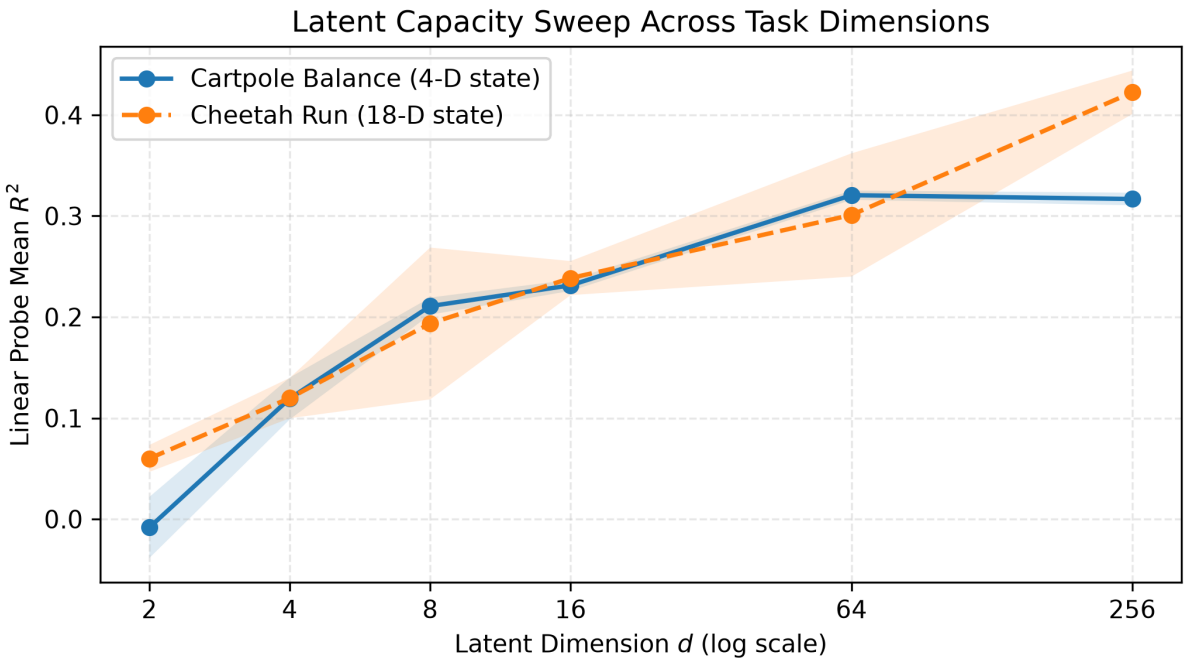


Figure 3.14 State-probe performance across latent dimensions

For Cartpole Balance, the mean probe R^2 begins at −0.008 for $d = 2$, rises to 0.119 at $d = 4$, and reaches 0.231 at $d = 16$. The strongest result in this sweep is 0.321 ± 0.005 at $d = 64$. Increasing the dimension to $d = 256$ does not improve it: the result is 0.317 ± 0.006. Under this configuration, performance has largely levelled off between 64 and 256 dimensions.

The result does not place the plateau at the task's four state variables. A four-dimensional output provides four real-valued coordinates, but that alone does not mean the encoder has organized those coordinates into a representation from which the physical state is easy to decode. At $d = 4$, the probe R^2 of 0.119 remains weak. The poor score at $d = 2$ is consistent with a restrictive representation losing useful state information, but this experiment does not explicitly identify pairs of physical states that have collided in latent space. The score alone is evidence of weak linear decodability, not a direct demonstration of a particular aliasing pair.

Cheetah Run behaves differently. Its mean probe R^2 rises from 0.060 ± 0.013 at $d = 2$, to 0.238 ± 0.017 at $d = 16$, and 0.301 ± 0.061 at $d = 64$. At $d = 256$, it reaches 0.422 ± 0.021, the best score measured for this task. Unlike Cartpole, Cheetah has not reached a clear plateau within the tested range.

These results support the direction of the hypothesis: the four-variable task approaches a plateau at a smaller dimension, while the eighteen-variable task continues to benefit from increased capacity through $d = 256$. They do not establish the location of Cheetah's eventual plateau. The sweep tests six dimensions, and its highest tested value is not necessarily sufficient.

The experiment is also not a controlled comparison of latent dimensions in which all else is mathematically identical. Each point comes from a newly trained encoder and projection head. The

result therefore reflects how the fixed architecture, contrastive objective, data, and training budget work at each output dimension. The evidence answers a practical question about this training setup, not a universal law connecting physical-state dimension to optimal latent size.

TIP — Choose the smallest dimension that has saturated. Start with a sweep rather than selecting d by intuition. Look for the smallest dimension at which the measured state probe stops improving meaningfully, and include variation across seeds. If performance is still rising at the largest tested dimension, the sweep has not yet identified a plateau.

SIDEBAR — The bottleneck in one paragraph. The latent dimension limits the coordinates available to the representation. Too little capacity can make task-relevant distinctions difficult to preserve; more capacity can improve decodability, but also costs computation and may not help the task. The right dimension is not automatically the state-vector length. It is the smallest practical representation that retains enough information for the work the agent must do, judged by measurements relevant to that work.

The distinction between reconstruction, invariance, and predictive information now becomes more concrete. Section 3.5 showed that reconstructing pixels does not guarantee that state is linearly available. This section shows that even a representation with more dimensions is useful only to the extent that it preserves the distinctions a probe can recover. In the next section, the question changes from which static observations should agree to which future observation follows from the past.

3.7 Predicting over time: from invariance to prediction

3.7.1 Why the future is not just another view

Return to the view-generation decision from Section 3.3. Two augmented versions of the same frame are positive examples because the objective asks the encoder to preserve information that survives those augmentations. The brightness and small crop changes are intended to be nuisance variations, so the learned representation should be relatively insensitive to them.

Now suppose we instead pair frames from times t and $t + \Delta$ as positives and ask their embeddings to agree. The scene may have changed during that interval. The cart may have moved, the pole may have rotated, and the object's velocity may have changed. An objective that rewards agreement between the two embeddings can treat these differences as information to suppress.

That may be undesirable for a world model. The future depends on the current situation and its dynamics. If the representation is encouraged to remove the differences that signal those dynamics, a downstream predictor has less useful information to work with.

This is not a claim that temporal contrastive learning must always discard dynamics. The result depends on how temporal positives are constructed and what else the objective asks the model to preserve. The narrower point is that invariance and prediction impose different requirements. Invariance says which observations should agree; prediction says which future observations are compatible with a given past.

Our measurements from the earlier sections make this distinction concrete. Table 3.2 showed that velocity was poorly decoded from the learned single-frame representation under the tested configuration. Stacking two frames did not reliably fix the problem under those same experimental conditions. These results do not prove that velocity is fundamentally absent from images or impossible to recover from short sequences. They show that the representation and training setup did not make it reliably linearly available.

A frame-level invariance objective has no direct reason to preserve velocity. A temporal objective can instead reward a context for retaining information that helps identify a future observation. CPC is one way to implement that change.

3.7.2 Contrastive predictive coding

CPC, introduced by van den Oord and colleagues, learns by predicting future representations rather than reconstructing future pixels. It encodes a sequence of observations into latent vectors, summarizes the available past in a context vector, and trains that context to identify the correct future latent among other candidates.

Let a sequence of observations be $x_1, x_2, \ldots, x_T$. A shared encoder f_θ maps each observation to a latent representation:

$$z_t = f_\theta(x_t).$$

A recurrent model summarizes the representations observed up to time t:

$$c_t = g(z_1, z_2, \ldots, z_t).$$

The context c_t may depend on the current representation and every earlier representation in the sequence. It must not depend on future representations $z_{t+1}, z_{t+2}, \cdots$. Otherwise, the task would leak information from the answer into the context used to select that answer.

We use a gated recurrent unit GRU for g. A GRU maintains a learned hidden state as it processes the sequence one step at a time. Its purpose here is to produce a compact context vector, not to guarantee that every relevant aspect of the past is retained. Whether the learned context retains useful information is an empirical question.

Scoring a candidate future

For each prediction horizon k, CPC learns a matrix W_k. It scores a candidate future latent using a bilinear compatibility function:

$$\exp\left(z_{t+k}^{\top} W_k c_t\right).$$

Here, z_{t+k} is the candidate representation at a future time, c_t is the context available at time t, and W_k is specific to horizon k. The matrix lets the model learn which relationships between context coordinates and future latent coordinates are useful for identifying a match.

The exponent turns the raw bilinear score into a positive compatibility value. As in equation 3.5, the

objective normalizes these values across candidates to obtain a probability distribution. Higher compatibility means that a candidate receives a larger share of the probability assigned by the context.

Consider a batch containing B sequences. For a context from sequence i, the correct target is the future latent from sequence i. Future latents from the other sequences in the batch act as negative candidates. At horizon k, define the raw score

$$\left(z_{t+k}^{(j)}\right)^{\top} W_k c_t^{(i)},$$

where i identifies the context and j identifies a candidate future. The probability assigned to the correct candidate is

$$\frac{\exp(s_{ii}^{(k)})}{\sum_{j=1}^{B} \exp(s_{ij}^{(k)})}.$$

The loss for that context and horizon is the negative log-probability of the correct match. Averaging across contexts and trained horizons gives the multi-horizon InfoNCE objective:

$$-\frac{1}{|\mathcal{J}|} \sum_{(i,\ t,\ k) \in \mathcal{J}} \log \frac{\exp(s_{ii}^{(k)})}{\sum_{j=1}^{B} \exp(s_{ij}^{(k)})},$$

where $\mathcal{J}$ is the set of valid context, sequence, and horizon combinations in the batch. This is equation 3.5 aimed at a different task. Instead of identifying which view belongs to the same observation, it identifies which future latent belongs to the current context.

The model is trained at several horizons. A one-step prediction head scores the latent at (t+1); a two-step head scores (t+2); a three-step head scores (t+3). Each head has its own matrix W_k. The context is shared, so its representation must support identification of future latents at each of these horizons.

The model does not directly regress the future latent with a squared-error objective. It does not take a predicted vector, feed that vector back into the model, and recursively generate a long sequence. It learns compatibility scores that rank the correct future above other candidates. Those are distinct tasks, and the evaluation must respect that distinction.

The implementation

In the repository, worldmodels/models/temporal_cpc.py defines TemporalCPC. Its forward method encodes a batch of image sequences, runs the GRU, and calculates cross-entropy over candidate future latents at each valid context and horizon.

Source: `worldmodels/models/temporal_cpc.py`. The full runner lives in `ch03/11_temporal_cpc.py`.

Listing 3.12 CPC context and compatibility scoring

```
z_seq = self.encode_sequence(x_seq)
c_seq, _ = self.gru(z_seq)

b, t, _ = z_seq.shape
cpc_sum = torch.zeros((), device=x_seq.device)
n_cpc = 0

for t_idx in range(0, t - self.k_steps):
    c_t = c_seq[:, t_idx]
    for k in range(self.k_steps):
        target = z_seq[:, t_idx + 1 + k]
        pred = c_t @ self.w_k[k].T
        logits = pred @ target.T
        labels = torch.arange(b, device=x_seq.device)
        cpc_sum = cpc_sum + F.cross_entropy(logits, labels)
        n_cpc += 1

return cpc_sum / n_cpc
```

The matrix multiplication produces a batch-by-batch score matrix. Row i contains the scores assigned by context i to every candidate future latent in that batch. The correct target lies on the diagonal, because context and future target come from the same sequence. Cross-entropy encourages the diagonal score to dominate its row.

The GRU is unidirectional. Although the implementation processes a tensor containing multiple time steps at once, the context output at index t is computed causally from the sequence up to that time step. Future latents are targets used to calculate the loss; they are not inputs to the context at t.

WARNING — Causality. A context used to predict the future may contain the current observation and earlier observations, but not the future target. If future frames influence the context, the model can identify the target using information that would not be available at prediction time.

What enters the experiment

The CPC experiment uses Cartpole Balance frames collected under a uniform random policy. The model receives sequences of images. It does not receive the actions that generated those sequences.

That omission matters. In a controlled physical system, the next state depends not only on the current state but also on the action applied and the system's dynamics. CPC here learns from a sequence generated by an unknown-to-the-model action stream. Its context must infer useful information from the visual history alone.

This experiment is therefore not a controlled comparison between action-conditioned and action-free prediction. It does not tell us how much actions would improve the results. The missing action input is a limitation of this setup and one reason the next chapter treats predictive representations and action-

conditioning as separate design questions.



Figure 3.15 artwork pending: Contrastive predictive coding identifies a future latent

Production placeholder, not final artwork. See `../images/CH03_F15_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

3.7.3 CPC evaluation: future retrieval and state probes

A low CPC loss means that the correct future latent receives high probability under the model's candidate distribution. But the loss alone is harder to interpret than a direct retrieval measurement. We therefore evaluate the trained compatibility scores on held-out sequence windows and ask whether the correct future latent ranks above the other candidates.

This evaluation is aligned with the objective CPC actually trains. It is not a physical-state rollout experiment.

Evaluation protocol

The committed run uses five trained model seeds: 0 through 4. Each model was trained for 20 epochs with a batch size of 32, learning rate 10^{-3}, encoder latent dimension 16, GRU context dimension 32, and three prediction horizons. The training configuration caps the data at 4,000 training sequence windows and 1,000 validation sequence windows, each of length 16 frames.

For the primary retrieval evaluation, the experiment selects 200 non-overlapping, episode-safe windows from the validation split. The selection is balanced across five validation episodes, with 40 windows per episode. The windows do not overlap within their source episodes, and they are shuffled into evaluation batches so that candidate sequences are not adjacent windows presented in temporal order. These 200 windows are not 200 independent trajectories; they come from only five validation episodes.

Evaluation compares each context's correct future latent with the corresponding future latents from other sequences in the same batch. There are six full batches of 32 candidates and a final batch of eight. The chance level is consequently computed for this query-weighted, in-batch protocol:

$$\frac{192(1/32) + 8(1/8)}{200} = 0.035 = 3.5\%$$

The random top-1 chance rate is therefore 3.5%, rather than $1/200$. The test does not rank each context against all 200 windows simultaneously. That distinction is part of the result, not a detail to omit from the caption.

Future-latent retrieval

The top-1 retrieval rate is the fraction of queries for which the matching future latent receives the highest score in its batch. Top-5 retrieval counts a match as correct if it appears among the five highest-scoring candidates. The positive-minus-hardest-negative margin measures the score gap between the

correct future and the most competitive incorrect candidate. Because the logits are raw bilinear scores, that margin is a score difference, not a probability.

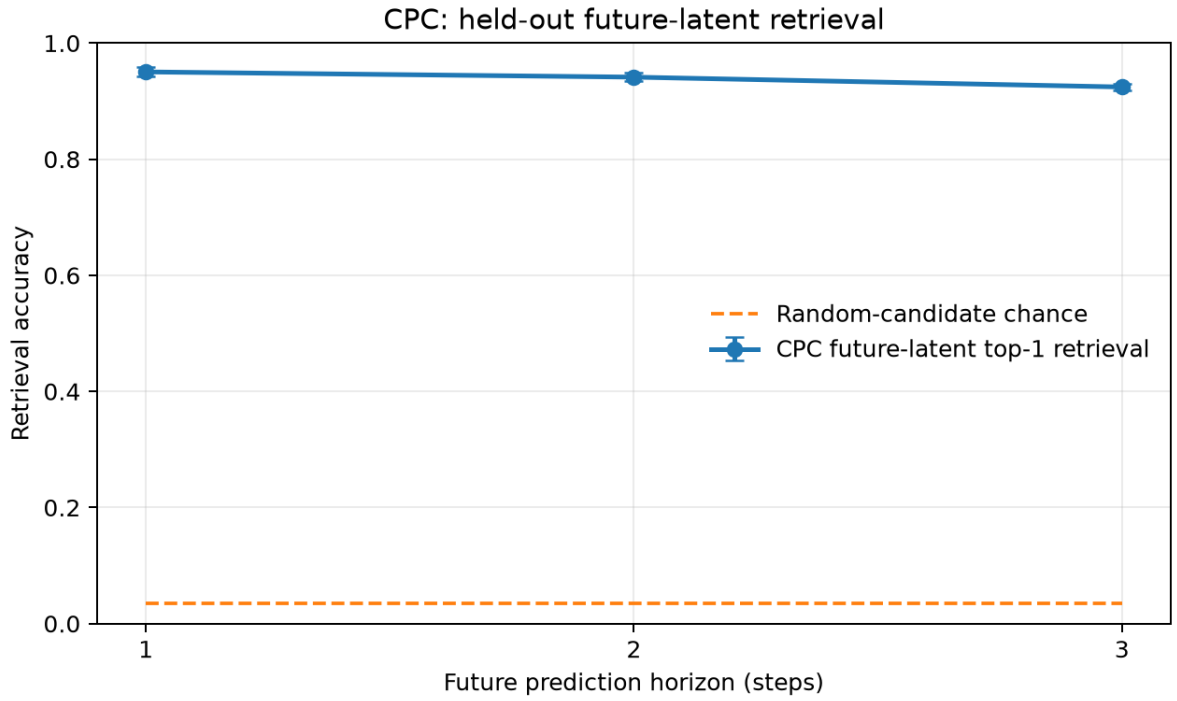


Figure 3.16 CPC future-latent retrieval by horizon

Panel A. Static-encoder and recurrent-context probes for current and next-state variables.

Table 3.4 CPC state probes and future-latent retrieval

State variable	Static $z_t \rightarrow s_t$	Context $c_t \rightarrow s_t$	Static $z_t \rightarrow s_{t+1}$	Context $c_t \rightarrow s_{t+1}$
Cart position	0.770 ± 0.033	0.805 ± 0.084	0.770 ± 0.033	0.806 ± 0.084
Pole angle	0.515 ± 0.037	0.560 ± 0.051	0.514 ± 0.039	0.559 ± 0.050
Cart velocity	0.013 ± 0.006	0.016 ± 0.006	0.014 ± 0.006	0.016 ± 0.006
Pole angular velocity	−0.347 ± 0.017	−0.348 ± 0.030	−0.345 ± 0.017	−0.346 ± 0.029

Panel B. CPC future-latent retrieval, InfoNCE loss, and positive–hardest-negative margin.

Future horizon	Top-1 retrieval	Top-5 retrieval	InfoNCE loss	Positive–hardest-negative margin
1 step	95.0% ± 0.8 pp	100.0% ± 0.0 pp	0.115 ± 0.016	12.96 ± 0.65
2 steps	94.1% ± 0.7 pp	100.0% ± 0.0 pp	0.133 ± 0.017	11.52 ± 0.54
3 steps	92.4% ± 0.5 pp	100.0% ± 0.0 pp	0.171 ± 0.018	9.50 ± 0.50

Note: Chance top-1 retrieval for this query-weighted in-batch evaluation is 3.5%. The uncertainty

shown is the standard deviation across the five trained seeds, not a confidence interval over 200 independent episodes.

Top-1 retrieval is high at all three horizons and declines from 95.0% at one step to 92.4% at three steps. InfoNCE loss rises with horizon, while the margin between the positive and hardest negative decreases. These metrics are consistent with identifying the matching future latent becoming less separated from other candidates at longer horizons, even though the correct candidate remains ranked first most of the time.

Top-5 retrieval reaches 100% in each seed at all three horizons. This tells us that the positive candidate reliably appears among the five highest-scoring candidates in the batches used for evaluation. It does not mean the model can predict five steps into the future with perfect physical accuracy.

The candidate protocol also limits the claim. Candidates come from only the selected validation windows and are compared within batches; the benchmark does not test retrieval across every possible future state or every possible trajectory. The five episodes contribute many windows, but windows from the same episode are not independent evidence of generalization to wholly new environments.

The recurrent context has a numerically higher current cart-position R^2 than the static encoder: 0.805 versus 0.770. Its mean next-state R^2 across the four state variables is also numerically higher, at 0.259 ± 0.016, compared with 0.238 ± 0.009 for the static representation. The per-variable measurements show that this change is limited. Cart velocity remains very weakly decodable, and pole angular velocity remains negative under both representations.

Negative R^2 means that the probe performs worse on the validation samples than a constant predictor based on the validation-target mean. It does not prove that the representation contains no information about that variable; it says the fitted linear readout does not recover it effectively under this evaluation.

The variation across seeds also matters. The context's cart-position score varies more from seed to seed than the static score, and the study does not establish statistical significance for the differences between the two representations. It would be too strong to conclude that adding a GRU solves state estimation or makes velocity available. The measurement supports a more limited result: the recurrent context changes what is linearly available, but the change is not uniform across physical variables.

What the experiment establishes

The primary result is that the trained CPC compatibility function identifies matching future latents substantially above the protocol's 3.5% random-candidate chance level. Its retrieval performance remains high over three trained horizons, while the loss and margin indicate somewhat less separation at longer horizons.

The state probes provide a complementary picture. Current position is relatively well represented, but velocity remains poorly decoded from both the static latent and the recurrent context. High future-latent retrieval therefore does not imply that every physical variable is explicitly available in the

representation. Nor does it establish that the model can predict a physical state recursively, remain stable across long rollouts, or support a useful planner.

The experiment also receives no action input. Because the frames were generated under a random policy, the learned context does not distinguish state-driven change from the effects of the actions taken. The study is not a controlled test of action-conditioning; it simply does not include it. We must not interpret the retrieval score as action-conditioned transition prediction or as a universal ceiling on what CPC can achieve.

WARNING — A GRU is a placeholder, not a memory guarantee. The context vector summarizes a finite sequence through a learned recurrent state. A GRU does not guarantee that information needed over long periods will be retained. The memory chapter returns to that question using an environment designed to test long-term memory, rather than treating this short-window retrieval task as a memory benchmark.

CPC provides a bridge between the objectives in this chapter and the predictive representations discussed next. Instead of enforcing agreement between two observations or rebuilding missing pixels, it asks a context to identify a future embedding. Chapter 4 develops the joint-embedding predictive idea further through JEPA architectures, with a different emphasis on learned prediction targets and, later, action-conditioning.

We now have several ways to train a visual representation. Contrastive learning encourages agreement across views. Non-contrastive objectives add mechanisms to prevent collapse. Masked prediction trains a model to reconstruct hidden pixels, and CPC asks a context to identify a future representation.

These methods differ in what they reward, but the agent should not need a different interface for every training objective. Once an encoder has been trained, the agent needs a component that accepts an observation and returns a representation. If the agent maintains a recurrent state, the same component must also update that state as new observations arrive.

This is where the chapter's work returns to the agent loop from Chapter 1. We will use a trained contrastive encoder behind a common interface. The dynamics and reward components remain placeholders; this section assembles the representation component rather than claiming to complete the world model.

3.8 Assembling the encoder

3.8.1 A common encoder interface

The current repository implements the shared interface in `worldmodels/models/encoders.py`. The Encoder class wraps a backbone, such as the ConvEncoder trained in Section 3.3. It exposes two methods:

`encode(obs)` maps an observation to its latent representation.

`update(state, obs)` processes the observation and returns the updated agent state.

The second method can operate in either of two ways. Without a recurrent layer, it simply returns the encoded observation. With a recurrent layer, it passes the representation and previous recurrent state through a GRU cell. On the first call, when no previous state exists, it initializes the hidden state to zeros.

Source: `worldmodels/models/encoders.py`. The excerpt below is taken from the canonical implementation.

Listing 3.13 The encoder interface

```
class Encoder(nn.Module):
    """Unified encoder interface used by the Chapter 1 agent loop."""

    def __init__(
        self,
        backbone: nn.Module,
        recurrent_hidden_dim: Optional[int] = None,
    ) -> None:
        super().__init__()
        self.backbone = backbone
        self.recurrent_hidden_dim = recurrent_hidden_dim
        if recurrent_hidden_dim is not None:
            latent_dim = getattr(backbone, "latent_dim", 16)
            self.gru_cell = nn.GRUCell(
                latent_dim, recurrent_hidden_dim
            )
        else:
            self.gru_cell = None

    def encode(self, obs: torch.Tensor) -> torch.Tensor:
        if obs.ndim == 3:
            obs = obs.unsqueeze(0)
        return self.backbone(obs)

    def update(
        self,
        state: Optional[torch.Tensor],
        obs: torch.Tensor,    ) -> torch.Tensor:
        z = self.encode(obs)
        if self.gru_cell is None:
            return z
        if state is None:
            state = torch.zeros(
                z.shape[0],
                self.recurrent_hidden_dim,
                device=z.device,
```

```
                dtype=z.dtype,
            )
        return self.gru_cell(z, state)
```

The wrapper keeps the representation-learning architecture separate from the agent's interface. The backbone determines how observations are encoded. The optional GRU cell determines how successive encodings are combined into a persistent hidden state.

The word state in this interface refers to the encoder's recurrent hidden state, not the simulator's ground-truth physical state. During normal operation, the agent does not receive qpos or qvel from the simulator. It receives the rendered observation and carries its own internal state between calls.

The wrapper also does not dictate how the encoder was trained. A different compatible backbone can be substituted without changing the calls made by the agent loop. The interface is small, but it creates an important separation: training objectives can change while the rest of the agent continues to consume an observation and an updated internal state.

3.8.2 Verifying the contract

The repository's ch03/12_assemble_encoder.py runner checks this interface using a saved contrastive encoder checkpoint. It loads the encoder with a latent dimension of 16 and wraps it with a recurrent hidden dimension of 32. The script then processes ten validation frames in order, passing the previous hidden state into each subsequent update.

The expected recurrent-state shape is `(1, 32)`. The committed output in ch03/results/table_03_06_synthesis.json records that shape and the ten frames tested.

This is an interface smoke test, not an evaluation of representation quality. It verifies that the checkpoint loads, the encoder processes observations, and the recurrent-state contract holds for the tested sequence. It does not establish that the representation is sufficient for control, that the recurrent state captures all relevant history, or that the assembled agent can act successfully in the environment.

The interface therefore completes a software contract, not the scientific contract from Chapter 2. Whether the representation is useful depends on the measurements from the preceding sections and on what the later dynamics and reward components require.

3.8.3 What each objective pays to preserve

The experiments show why a common interface does not imply that all encoders learn equivalent representations. The objective used during training determines which relationships are rewarded. Table 3.5 summarizes the central trade-off from this chapter.

Table 3.5 Representation objectives and their trade-offs

Objective	Paid to keep	May discard	Invited failure
Pretext tasks	Task-defined structure	Variables irrelevant to the pretext	Shortcut features
Contrastive (NT-Xent)	Structures preserved across views	Information removed by augmentation	False negatives and aggressive invariance
Non-contrastive methods	Invariant, non-collapsed coordinates	Redundant coordinates	Collapse if the anti-collapse mechanism fails
Masked autoencoding	Information useful for reconstruction	Details irrelevant to pixel recovery	Capacity spent on predictable pixels
Temporal CPC	Information useful for future prediction	Unpredictable innovations	Action-unconditioned prediction ceiling

The table is not a ranking. Each row describes the direct pressure imposed by a family of objectives, together with a corresponding risk. Whether a particular model actually suffers that failure must be measured.

Pretext tasks define their own prediction target. That target can create useful supervision from the structure of observations, but it can also reward shortcuts that solve the pretext without preserving the information needed downstream.

Contrastive learning pays for agreement across the views constructed by the augmentation pipeline. This is useful when the chosen transformations preserve task-relevant distinctions. It becomes a problem when an augmentation removes information the agent needs, or when different frames that look similar become false negatives.

Non-contrastive methods demonstrate that negatives are not the only way to prevent collapse. SimSiam, BYOL, Barlow Twins, and VICReg use different mechanisms to constrain the learned representation. Those mechanisms introduce their own design choices, and the reduced-scale experiments in this chapter do not establish a universal ranking of the methods.

Masked autoencoding pays for reconstructing missing pixel content. It may devote capacity to background or other predictable image details that do not make physical variables easy to decode. The Cartpole experiment found that reconstruction quality and state-probe quality are distinct measurements.

CPC changes the target from reconstructing an observation to identifying its future representation from a context. The retrieval experiment shows that the correct future latent can be identified at high rates within the tested candidate batches. Its state probes remain weak for velocity, and the model receives no action input. Future-latent identification alone is therefore not a guarantee of a complete predictive state.

The common pattern is that an objective cannot be judged solely by whether its loss decreases. We need to examine what its representation preserves, what it discards, and whether those properties

match the agent's requirements.

3.8.4 Checking the Chapter 2 contracts

The measurements in this chapter give partial evidence about the contracts introduced in Chapter 2. They do not settle every question about the learned state. The following checklist separates what was measured from what remains unverified.

Chapter 2 contract check

- **Sufficiency: preserve information relevant to the task** Evidence: Frozen-feature linear probes measure whether physical state variables are linearly recoverable. Position variables are often more decodable than velocities. The MAE, contrastive, and VICReg comparison also shows that results differ across complete model configurations. Remaining limit: A linear probe measures explicit linear decodability, not all information present in a representation. These experiments do not demonstrate successful control or planning.
- **Minimality: avoid retaining unnecessary detail** Evidence: The latent-dimension sweep measures probe performance from $d = 2$ through $d = 256$. Cartpole approaches a plateau between $d = 64$ and $d = 256$, while Cheetah performance is still rising at $d = 256$. Remaining limit: The sweep does not measure mutual information directly, prove a globally minimal representation, or establish a plateau for Cheetah.
- **Aliasing: distinguish situations that require different predictions or actions** Evidence: The aliasing diagnostic compares single-frame and stacked-frame representations. Mean cart-velocity R^2 changes only from approximately 0.006 to 0.009, while pole-angular-velocity R^2 becomes slightly worse, from approximately −0.367 to −0.375. Remaining limit: Under the tested resolution, augmentation, architecture, and budget, stacking two frames did not reliably recover velocity. This does not prove that temporal information can never resolve the ambiguity.
- **Markov property and predictive state: retain history relevant to the future** Evidence: CPC's GRU context identifies matching future latents with top-1 retrieval of approximately 92%–95% across the three tested horizons. The mean next-state probe R^2 is numerically higher for the context than for the static encoder. Remaining limit: The context does not make every physical variable linearly decodable. The experiment does not establish conditional independence from earlier history, accurate recursive physical prediction, or stable long-horizon rollouts.
- **Action-conditioned dynamics: predict outcomes in relation to actions** Evidence: The CPC experiment measures future-latent identification using sequences collected under a random policy. Remaining limit: The model receives frames but not actions. This experiment does not evaluate action-conditioned prediction or establish planning utility.



Figure 3.17 artwork pending: The learned encoder in the agent loop

Production placeholder, not final artwork. See `../images/CH03_F17_Dadwal.md` for the required composition, source assets, credits, and acceptance checks.

Exercises

These exercises connect the chapter's mathematical objectives to the behavior of the representations they train. For each experiment, write down a hypothesis before running the code, preserve the configuration and random seeds, and distinguish a measured result from an explanation of why it occurred.

1. Deriving the contrastive gradient

Starting from equation 3.5, derive the gradient of the NT-Xent loss with respect to the anchor representation z_1.

Separate the gradient into the contribution from the positive example and the contributions from the negative examples. Identify which terms pull the positive closer and which push negative candidates away. Explain how the temperature τ changes the relative weights assigned to candidates.

2. Temperature and negative-example weighting

Consider InfoNCE as the temperature approaches infinity, $\tau \to \infty$. Show what happens to the softmax probabilities when the similarities are divided by an increasingly large temperature.

Predict the effect on the gradients and on the learned representation. Then increase the temperature in the contrastive training experiment and compare the loss curve and frozen-feature state probes against a baseline. Keep the data, architecture, and training budget fixed.

3. Removing a VICReg term

VICReg combines invariance, variance, and covariance penalties. Which term prevents the trivial constant representation from satisfying the objective at zero cost? What failure would you expect if the covariance penalty were removed while the other terms remained?

State your predictions for latent spread, effective rank, and state-probe performance. Then modify the experiment in ch03/08_vicreg.py, run it with the same seeds and budget, and compare the measurements with the complete objective. Distinguish a reduction in redundancy from a genuine collapse.

4. When augmentation removes state information

The contrastive view pipeline uses crop shifts of up to three pixels. Suppose the maximum shift increases to 16 pixels, while Cartpole remains rendered at 64×64 resolution.

Predict how this change will affect the cart-position probe and explain your prediction using the chapter's central principle: the objective rewards information preserved by the view pipeline. Then run the experiment with the larger shift, keeping all other settings fixed. Report the cart-position score across seeds and compare it with the baseline.

5. Choosing the latent dimension

The bottleneck sweep measures state-probe performance at latent dimensions $d \in 2, 4, 8, 16, 64, 256$ for Cartpole Balance and Cheetah Run.

At what dimension does Cartpole's probe approach a plateau? Does Cheetah show the same pattern within the tested range? Relate the results to the tasks' four- and eighteen-dimensional physical states, respectively, and explain why the optimal latent dimension need not equal the number of state variables.

Use Figure 3.14 and ch03/results/bottleneck_sweep_metrics.json. Identify the smallest tested dimension that appears to approach saturation for each task, and state what further experiment would be needed if performance is still rising at the largest dimension.

6. Extending CPC's prediction horizon

Modify the CPC experiment to predict five future steps instead of three. Predict how future-latent retrieval, InfoNCE loss, and the positive-versus-hardest-negative score margin will change as the horizon increases.

Train fresh checkpoints with five horizon-specific scoring matrices. Use separate checkpoint, result, and figure paths so the canonical Table 3.4 artifacts remain unchanged. Compare the new results with the three-horizon baseline using the same evaluation protocol.

Explain the difference between identifying the correct future latent among candidates and predicting a physical state accurately. In particular, state why a strong retrieval result does not establish accurate recursive prediction, action-conditioned dynamics, or planning utility.

7. Testing temporal aliasing on Finger Spin

The existing aliasing diagnostic compares single-frame features with features formed from two consecutive Cartpole frames. Its current runner hardcodes the Cartpole datasets and four Cartpole state-variable names. Extend it to support Finger Spin before running the experiment, including the correct variable mapping and the periodicity-aware evaluation for the spinner's angle.

Predict which Finger Spin state variables will be easier to decode from a single frame and which, if any, will benefit from stacking two frames. Then compare single-frame and stacked-frame probe scores across the same three seeds.

> **NOTE: Interpreting Finger Spin probes.** Finger Spin's rendered spinner has π-periodic symmetry: a static image does not uniquely identify its absolute 2π angle. Account for this when interpreting the probe scores. Distinguish a variable that is unobservable from a single image because of the rendering symmetry from one that is merely difficult to decode under the tested model and training budget.

Summary == Summary * Pretext tasks: Rotation prediction, jigsaw prediction, and colorization create

supervision from the structure of observations. Their limitation is that solving a pretext task does not guarantee that the representation preserves the information needed by a world model. The chosen prediction target determines what the model is paid to learn. * Contrastive learning: NT-Xent turns the identification of matching views into a classification problem, using similarities between positive and negative examples. On Cartpole, the training loss fell from approximately 3.78 to 3.11 over 15 epochs across the three-seed run. State probes showed that positions were more linearly accessible than velocities, and stacking two consecutive frames did not reliably improve velocity decoding under the tested configuration. * Collapse and its remedies: An attraction-only objective has a trivial constant-representation solution, so agreement between views is not enough. SimSiam, BYOL, Barlow Twins, and VICReg introduce different mechanisms to avoid collapse or constrain redundancy. Their effectiveness must be assessed using more than the training loss alone, including feature spread, effective rank, and downstream probes. * Masked prediction: A masked autoencoder hides 75% of image patches and predicts their pixel values, with reconstruction loss applied only to the masked patches. In the standalone Cartpole experiment, its frozen-feature probe achieved mean R^2 of 0.170 ± 0.004 across three seeds. Reconstruction and state decodability are different measurements, and the method-family comparison is not a controlled ablation of the loss alone. * Latent bottlenecks: The sweep found that Cartpole's mean probe R^2 rose from approximately -0.008 at $d = 2$ to 0.321 at $d = 64$, then remained similar at $d = 256$. Cheetah's probe R^2 continued rising to 0.422 at $d = 256$. The smallest useful representation depends on the task and the measured plateau; it cannot be determined simply by counting physical state variables. * Prediction over time: CPC uses a causal GRU context and multi-horizon InfoNCE to identify the correct future latent among candidates. Top-1 retrieval reached 95.0%, 94.1%, and 92.4% at horizons one, two, and three, respectively, against a 3.5% query-weighted in-batch chance level. These results demonstrate future-latent identification under the tested protocol, not complete state recovery, action-conditioned dynamics, stable physical rollouts, or planning utility. * Assembling the encoder: A common `encode(obs)` and `update(state, obs)` interface connects the learned visual encoder to the Chapter 1 agent loop, while dynamics and reward remain to be implemented. Across all these methods, the governing principle is that a representation-learning objective is a specification of what information the representation is paid to keep. == Appendix A: InfoNCE bound and gradient details This appendix contains the algebra moved out of §3.3 to preserve the main chapter's teaching flow while retaining the full derivation. It uses the same notation as Table 3.1. The claims are conditional on the sampling assumptions described in §3.3; the recorded trajectory batch is not an ideal independent-negative sampler.

Where the bound comes from

Sidebar: A proof of the InfoNCE bound

Suppose one of $K+1$ candidates is drawn from the conditional distribution $p(b \mid a)$, while the others are drawn independently from the marginal distribution $p(b)$. Let $r_j = p(b_j \mid a) / p(b_j)$, and let $S = \sum_{j=0}^{K} r_j$.

Step 1: The best possible classifier. Given the anchor and candidates, Bayes' rule assigns candidate j probability r_j / S of being the positive. Any classifier q_j incurs cross-entropy at least as large as this true posterior:

$$\mathcal{L}(f) \geq -\mathbb{E} \log \frac{r_m}{S}.$$

Step 2: Express that cross-entropy using mutual information. The expected log likelihood ratio of a positive pair is $I(a; b)$, so the expression on the right is

$$\mathbb{E}\log S - I(a; b).$$

Step 3: Bound the remaining term. Under the reference distribution where all $K+1$ candidates are independently sampled from the marginal $p(b)$, write $T = S/(K+1)$. The actual candidate-set distribution has density T relative to this reference distribution, so

$$\mathbb{E}[\log S] = \log(K+1) + \mathbb{E}_0[T\log T] \geq \log(K+1),$$

because $\mathbb{E}_0[T=1]$ and $t\log t$ is convex, so Jensen's inequality makes $\mathbb{E}_0[T\log T]$ nonnegative. Combining the three steps yields

$$\mathcal{L}_{\text{InfoNCE}} \geq \log(K+1) - I(a; b),$$

which rearranges to equation 3.6.

The gradient can also be made precise. The result in equation 3.7 separates attraction toward the positive from repulsion by negatives. For an anchor i, define the score $s_{ij} = (z_i, z_j)$. Differentiating the cross-entropy in equation 3.5 with respect to a candidate score gives

$$\frac{p_i(j) - \mathbf{1}[j = i^+]}{\tau},$$

where $\mathbf{1}[\cdot]$ equals one if its condition is true and zero otherwise. Applying the chain rule yields the gradient with respect to the anchor representation:

For cosine similarity,

$$\frac{1}{\| u \|_2}\left(\frac{v}{\| v \|_2} - (u, v)\frac{u}{\| u \|_2}\right).$$

The gradient is perpendicular to u, so each score's local update changes the direction of the vector rather than its length to first order.

Why this bound is not a physical-state guarantee

The lower bound concerns mutual information shared by paired-view representations under the stated sampling assumptions. It does not establish that cart position, angle, velocity, or an action-conditioned predictive state is linearly available. The data-processing inequality also limits shared information to information that survives the view-generation pipeline. These are the precise reasons the chapter follows the training objective with held-out state probes. == References and figure credits === Benchmark and environment

- Tunyasuvunakool, S. et al. (2020). "dm_control: Software and tasks for continuous control." *Software Impacts*, 6, 100022. https://doi.org/10.1016/j.simpa.2020.100022
- Google DeepMind, `dm_control` source repository: https://github.com/google-deepmind/dm_control

- MuJoCo documentation: https://mujoco.org/

Self-supervised objectives

- Gui, J. et al. (2024). "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12), 9052–9071. https://doi.org/10.1109/TPAMI.2024.3415112. Preprint: https://arxiv.org/abs/2301.05712
- LeCun, Y. (2020). "Self-Supervised Learning." ICLR 2020 keynote. Official session page: https://iclr.cc/virtual_2020/speaker_7.html. Public video recording: https://www.youtube.com/watch?v=8TTK-Dd0H9U.
- Chen, T. et al. (2020). "A Simple Framework for Contrastive Learning of Visual Representations." *ICML*. https://arxiv.org/abs/2002.05709
- van den Oord, A., Li, Y., and Vinyals, O. (2018). "Representation Learning with Contrastive Predictive Coding." https://arxiv.org/abs/1807.03748
- Chen, X. and He, K. (2021). "Exploring Simple Siamese Representation Learning." *CVPR*. https://arxiv.org/abs/2011.10566
- Grill, J.-B. et al. (2020). "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning." *NeurIPS*. https://arxiv.org/abs/2006.07733
- Zbontar, J. et al. (2021). "Barlow Twins: Self-Supervised Learning via Redundancy Reduction." *ICML*. https://arxiv.org/abs/2103.03230
- Bardes, A., Ponce, J., and LeCun, Y. (2022). "VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning." *ICLR*. https://arxiv.org/abs/2105.04906
- He, K. et al. (2022). "Masked Autoencoders Are Scalable Vision Learners." *CVPR*. https://arxiv.org/abs/2111.06377

Implementation and reproducibility

- Canonical implementation modules: `worldmodels/` in the code companion.
- Experiment runners, outputs, and numerical provenance: `ch03/` and `ch03/results/`.
- Current figure assets: `ch03/figures/`.
- Exact runtime and rendering guidance: `ch03/README.md`. Hardware-dependent runtime estimates must be reported as configuration- and machine-specific, not guaranteed speedups.

Figure credits

Figure 3.1 is an original objective-centred redraw informed by Gui et al. (2024), Uelwer et al. (2025), and the V-JEPA 2 release (2025). Figure 3.4 adapts the conceptual pretext-task overview from Gui et al. (2024). Figure 3.10 adapts the high-level Siamese-method comparison from Gui et al. (2024). Figure 3.12's MAE architecture panel is adapted from Gui et al. (2024) and informed by He et al. (2022); its reconstruction panel comes from the committed experiment output. Figure 3.13 adapts the conceptual distinction between joint embedding and pixel reconstruction from Gui et al. (2024). Figures 3.2, 3.5,

3.6, 3.11, 3.15, and 3.17 are original diagrams. The DMC observations in Figure 3.3 are the book's own renders of the simulator. Figures 3.7, 3.8, 3.9, 3.11's curves, 3.12's reconstructions, 3.14, and 3.16 derive from committed experiment outputs; unfinished outputs are documented by same-stem Markdown briefs in `images/`.

Updated taxonomy sources

- Uelwer, T., Robine, J., Wagner, S. S., Höftmann, M., Upschulte, E., Konietzny, S., Behrendt, M., et al. (2025). "A survey on self-supervised methods for visual representation learning." **Machine Learning**, 114, Article 111. https://doi.org/10.1007/s10994-024-06708-7
- Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., et al. (2025). "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning." Meta AI research publication, June 11, 2025. https://ai.meta.com/research/publications/v-jepa-2-self-supervised-video-models-enable-understanding-prediction-and-planning/

[1] Tunyasuvunakool, S. et al. (2020). "dm_control: Software and tasks for continuous control." *Software Impacts*, 6, 100022. https://doi.org/10.1016/j.simpa.2020.100022.